

g·NEED ACCESS

NETWORK ENABLED EASY
DATA ACCESS



INSTRUCTIONS FOR USE



NETWORK API



User Manual: g.NEEDaccess Network API
Version 1.24.00 | Revision 1.0 | 2024

Copyright © g.tec medical engineering GmbH
www.gtec.at | office@gtec.at

CONTENTS

1	THE NETWORK API.....	4
1.1	BEFORE USING THE NETWORK API.....	4
1.2	TYPES OF NETWORK API MESSAGES	4
1.2.1	THE ANATOMY OF A NETWORK API CONTROL MESSAGE.....	5
1.2.2	THE HEARTBEAT MESSAGE.....	11
1.2.3	THE ACKNOWLEDGE MESSAGE.....	11
1.2.4	ASYNCHRONOUS EVENT MESSAGES.....	11
1.2.5	DATA TRANSMISSION.....	11
1.3	THE MESSAGE FLOW	12
1.3.1	THE MICRO MESSAGE FLOW	12
1.3.2	CONTROL MESSAGE REPLY (FROM THE SERVER).....	13
1.3.3	THE MACRO MESSAGE FLOW.....	13
2	THE GDSNETWORKAPIDEMO PROJECT	18
2.1	COMPILING THE GDSNETWORKAPIDEMO PROJECT ON LINUX.....	18
2.2	COMPILING THE GDSNETWORKAPIDEMO PROJECT ON WINDOWS.....	18
2.2.1	RUNNING THE EXECUTABLE.....	19
2.2.2	READING THE RECORDED DATA FILE.....	19
3	NETWORK API REFERENCE.....	20
3.1	COMMON FUNCTIONALITY	20
3.1.1	ERROR CODES.....	20
3.1.2	STATUS CODES.....	22
3.1.3	CONTROL MESSAGES.....	23
3.1.4	ASYNCHRONOUS EVENT MESSAGES.....	56
3.2	DEVICE SPECIFICS (GENERAL)	69
3.2.1	THE DEVICE CONFIGURATION.....	69
3.2.2	DEVICE-SPECIFIC CONTROL MESSAGES.....	69
3.2.3	DEVICE-SPECIFIC EVENT MESSAGES.....	71
3.3	G.USBAMP SPECIFICS	72
3.3.1	INTRODUCTION TO THE G.USBAMP CONFIGURATION	72
3.3.2	THE G.USBAMP DEVICE CONFIGURATION	73
3.3.3	CONTROL MESSAGES	80
3.4	G.HIAMP SPECIFICS	118
3.4.1	INTRODUCTION TO THE G.HIAMP CONFIGURATION	118
3.4.2	CONTROL MESSAGES	124
3.5	G.NAUTILUS SPECIFICS	167
3.5.1	INTRODUCTION TO THE G.NAUTILUS CONFIGURATION.....	167
3.5.2	CONTROL MESSAGES.....	172
3.5.3	ASYNCHRONOUS EVENT MESSAGES.....	239

1 THE NETWORK API

1.1 BEFORE USING THE NETWORK API

To be able to communicate over the network, the operating system or a third-party library provides commands for opening TCP/IP sockets, establish connections between them and for sending and receiving data.

The TCP protocol uses Nagle's algorithm by default to reduce overhead traffic by but may introduce delays of about 200 milliseconds for short messages. That delay may mess up timing during data acquisition crucially. Therefore, Nagle's algorithm must be disabled at both client and server sides to ensure proper timing.



IMPORTANT

Disable Nagle's algorithm used by TCP after opening a socket but before connecting the socket and right after accepting an incoming connection at both client and server sides. Do so by setting and enabling the `TCP_NODELAY` socket option. Consult the API reference of your operating system or the socket library you are using for details on how to accomplish that.

1.2 TYPES OF NETWORK API MESSAGES

The following definitions clarify the distinctions between control messages, asynchronous event control messages, measurement data transmissions, heartbeat, and acknowledge messages.

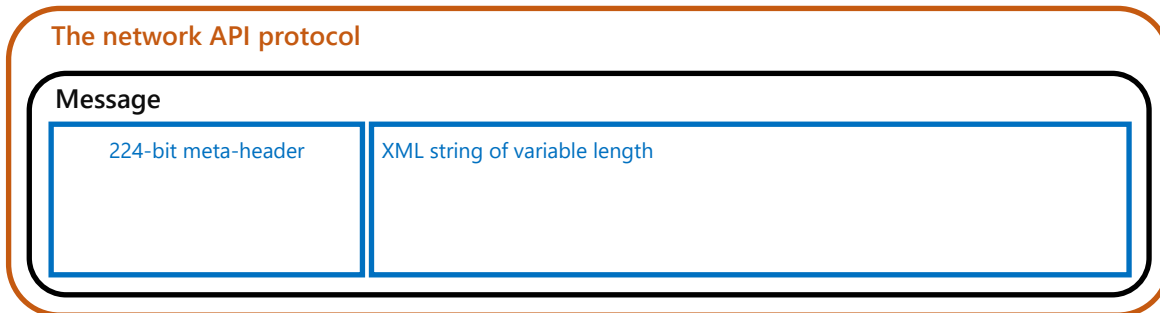
- **Control messages** are usually issued by the client as requested, and answered by the server with a reply. The request and the reply both consist of a meta-header, followed by an XML string of variable length containing the message content. The server provides exactly one socket (i.e. port) for control messages that all clients can issue their requests to.
- **Asynchronous event control messages** are triggered by the server without a preceding request from the client. Like control messages, they consist of a meta-header, followed by an XML string of variable length containing the event content.
- **Measurement data transmissions** consist of a meta-header, followed by raw measurement data in binary format. After the client requests data via a control message, data (meta-header + raw measurement data) is transmitted over a separate TCP/IP socket. For each DAQ session, the server provides a separate socket for data transmission. Please consult the g.NEEDaccess Client API documentation for disambiguation and details on the term DAQ session.
- **Acknowledge messages** only consists of a meta-header with the acknowledge flag set. An endpoint, whether server or client, must confirm each validly received message immediately with an acknowledge message. Hence, the server confirms an incoming control request from the client with an acknowledge message first, before it sends back an eventual reply that the client must then confirm with an acknowledge message.

- **Heartbeat messages** are issued periodically to each client from the server to check whether the client is still alive. A heartbeat message from the server only consists of a meta-header, with the acknowledge flag not set. The client has to acknowledge reception with an acknowledge message. It can be seen as a special asynchronous message.

1.2.1 THE ANATOMY OF A NETWORK API CONTROL MESSAGE

A network API message consists of a fixed size binary header (the meta-header), followed by an XML string of variable length containing the message's content.

The following figure shows the network API message.



The 224-bit meta-header

When data is received by g.NEEDaccess server, the meta-header (see

is used to decide whether the data comply with the network API standard or not. Therefore, the first 64 bits must contain the header identifier `0x3E47534D5344473C`, which corresponds to the ASCII text `<GDSMSG>` in reverse byte order. Values which are different from that result in an error.

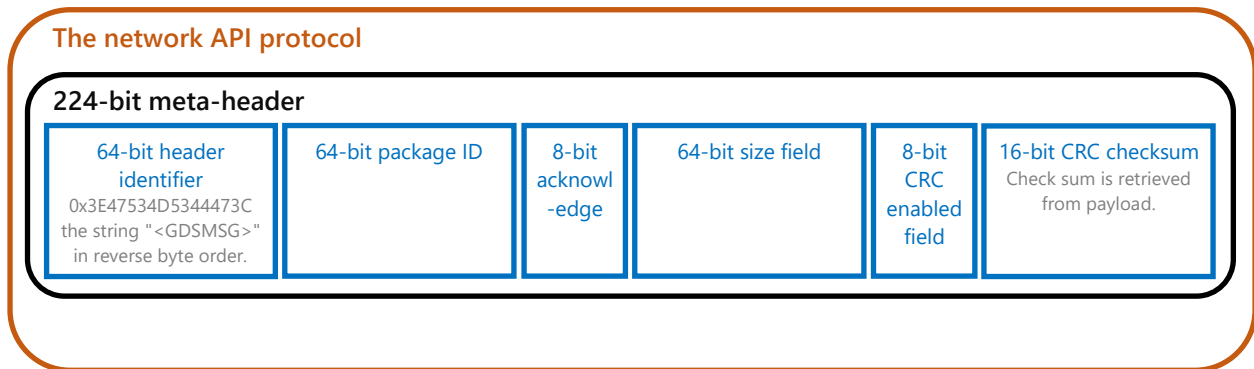
The next 64 bits hold the package identifier (ID). The ID is a consecutive number starting with zero. This is followed by an 8-bit acknowledge field, which can be either 0 or 1. The acknowledge field is zero for all message types except for acknowledge messages. These are header-only messages with acknowledge flag set. The next 64 bits are expected to hold the size of the XML string.

The next 8 bits hold the cycle redundancy check (CRC) enable field. The field is set to 0 if a CRC should not be applied on the XML string. Otherwise, it is set to 1. Values different from 0 or 1 do not comply with the network API standard and lead to an error.

If the CRC enable field is 1, the next 16 bits hold the CRC sum of the XML string where the XML string is interpreted as a raw array of bytes. For incoming messages, g.NEEDaccess server computes the CRC sum of the XML string and compares it with the CRC sum retrieved from the meta-header. If the CRC sums do not match, or if the CRC enable field is 0 while the CRC sum is not 0, an error will be triggered.

An error results in a cancellation of all asynchronous operations on the socket, and the socket is set back to the listening mode.

The following figure shows the 224-bit network API meta-header.



The XML string

This section follows a bottom up strategy to derive the architecture of the XML string that is used to interact with the services provided by the g.NEEDaccess server.

The interaction between the g.NEEDaccess server and a client is based on 5 fundamental data fields:

- An integer field (up to a range of 64 bits), which corresponds to the session ID. On the client side, the term "connection handle" is equivalent to the term "session ID". For a more detailed view on the term "connection handle", please consult the g.NEEDaccess Client API documentation.
- An integer field (up to a range of 32 bits), which corresponds to a command. See sections *3.1.3 Control messages* and *3.1.4 Asynchronous event messages* for possible command values.
- A payload field, which accompanies the command of interest. The payload is also coded as an XML string, but must be handled as inner text of the enclosing XML string. Therefore, the XML payload part must be encoded in escape style. The concrete type of the payload is interpreted in the context of the given command. Also, see section *Escaping the payload* on how to escape and handle an XML string in the payload field.
- An integer field (up to a range of 32 bits), which corresponds to the status of processing the command. See section *3.1.2 Status codes* for valid status codes.
- An integer field (up to a range of 32 bits), which holds an error code. See section *3.1.1 Error codes* for valid error codes.

The C++ programming language was used to build g.NEEDaccess. A network API control message is represented as an object of a certain class within g.NEEDaccess server. The fundamental data fields embody themselves as member variables of this class. The semantic of the data fields lead to the variable names, followed by their data type:

```
1. session_id_ : uint64_t
2. command_   : enum
3. payload_   : std::string
4. status_    : enum
5. error_     : uint32_t
```

The [boost::serialization](#) library is used to facilitate the transmission of a C++ object. The object can be represented as a XML expression, and this expression can be stored in a `std::string`. The procedure of serialization leads to the following XML element tags:

1. `<session_id_>VALUE</session_id_>`
2. `<command_>VALUE</command_>`
3. `<payload_>VALUE</payload_>`
4. `<status_>VALUE</status_>`
5. `<error_>VALUE</error_>`

These element tags are encapsulated by the parent element tags:

```
<value class_id="0" tracking_level="0" version="0">
    ...
</value>
```

which in turn are encapsulated by:

```
<boost_serialization signature="serialization::archive" version="19">
    ...
</boost_serialization>.
```

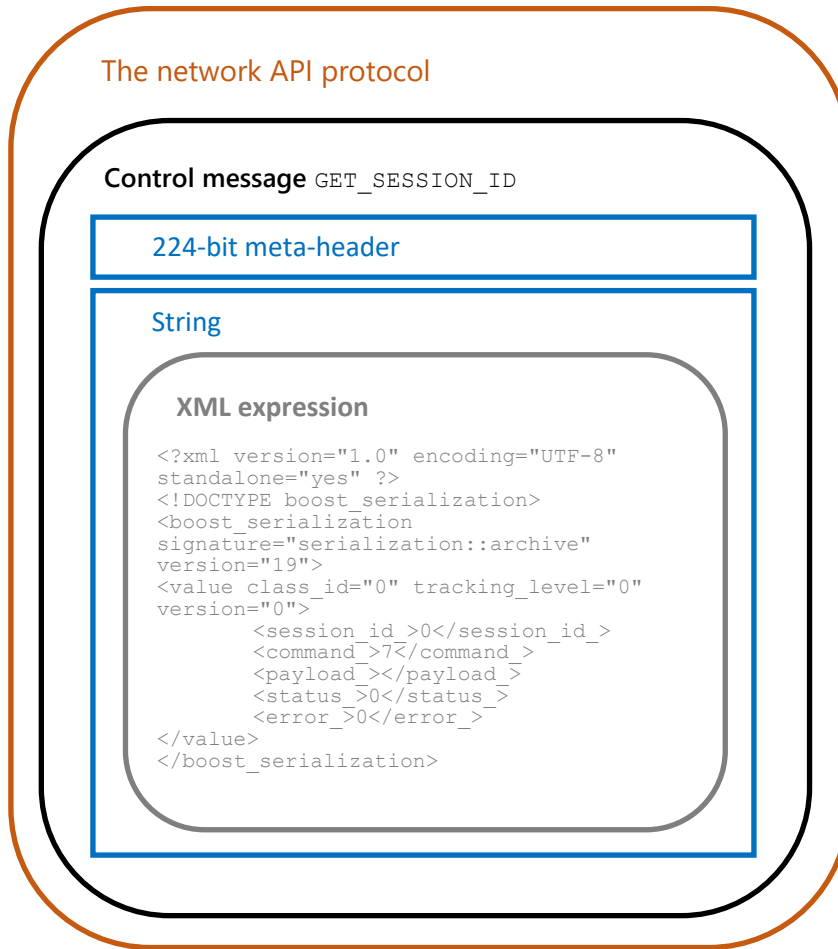
The document definition type (DTD) is mandated by boost and defined as:

```
<!DOCTYPE boost_serialization>
```

The XML expressions generated by boost::serialization always begin with the XML declaration:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
```

The result is the XML string representation of the message that is transmitted. The following figure shows an XML expression that requests creation of a new session ID for a client from g.NEEDaccess server (the server's reply is not included).



Escaping the payload

As mentioned above, the payload in the detailed network API message description is also given as an XML string for readability. But, since it is an inner text of the enclosing XML string, the payload's XML notation must be escaped before it can be passed on to the message when not using the [boost::serialization](#) library, which would do that automatically.

Escaping is the process of replacing particular characters that represent special meaning with alternative representation. There are five such special characters that must not appear in the payload section of the message. They're listed in the following table, together with their replacement string.

Non-escaped notation	escaped notation
&	&
"	"
'	'
<	<
>	>

**ATTENTION**

The & characters in the escaped notation must not be replaced by &.

An example in the detailed description of a network API message, the following excerpt of the description may appear:

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>30</command_>
    <payload_>
      </payload_>
    </payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <first>127.0.0.1</first>
    <second>51082</second>
  </value>
</boost_serialization>
```

**NOTE**

This XML expression corresponds to a serialized C++ std::pair<std::string, uint16_t>.

The *Client request* describes the XML string that follows the meta-header for a specific message. The payload field is described separately in non-escaped XML syntax. What we have to do now is escape the payload (i.e. replace all characters in the non-escaped notation column of the table with their escaped notations) and insert this into the payload field. The escaped payload will then read:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="9">
<value class_id="0" tracking_level="0" version="0">
<first>127.0.0.1</first>
<second>51082</second>
</value>
</boost_serialization>
```

After inserting this into the payload field, the total resulting client request that can be sent to the server will read:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id>1</session_id>
    <command>30</command>
    <payload>
      <?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
      <!DOCTYPE boost_serialization>
      <boost_serialization
signature="serialization::archive"
version="9">
        <value class_id="0"
tracking_level="0" version="0">
          <first>127.0.0.1</first>
          <second>51082</second>
        </value>
      </boost_serialization>
    </payload>
    <status>1</status>
    <error>0</error>
  </value>
</boost_serialization>
```

In turn, a reply from the server that contains a payload comes in escaped XML syntax, and must be decoded to non-escaped syntax before it will match the description. To do so, replace all occurrences of each string in the escaped notation column of the table with their non-escaped notation.

This holds true for all messages that have an XML string that has content in its payload field appended to the meta-header.

1.2.2 THE HEARTBEAT MESSAGE

The heartbeat message does not contain an XML string after the meta-header like the network API XML control message. The heartbeat consists of a blank meta-header only, with acknowledge flag not set, which is sent to all connected clients in a certain interval of time. If the process of sending a heartbeat message fails or the server does not receive a corresponding acknowledge message for an issued heartbeat message, the connection will be forcibly closed by the server, assuming the client is dead.

1.2.3 THE ACKNOWLEDGE MESSAGE

The acknowledge message does not contain an XML string after the meta-header like the network API XML control message. The acknowledge message consists of a meta-header only with acknowledge flag set. Additionally, the package ID of the acknowledge message's meta-header must correspond to the package ID of the message that it should acknowledge.

1.2.4 ASYNCHRONOUS EVENT MESSAGES

Asynchronous event messages are issued by the server to notify connected clients about important occurrences or incidents. They are similar to control message replies, except that the client did not issue a corresponding request before. The server may send them anytime to a connected client as appropriate. An asynchronous event message contains the session ID it belongs to as well as the type of the event as command.

1.2.5 DATA TRANSMISSION

After the client requests data by a control message, data is transmitted over a separate TCP/IP socket. One data package is made up of the meta-header, followed by binary formatted measurement data. Measurement data is a concatenation of 32-bit floating point values. The meta-header contains the size of the binary formatted measurement data in bytes.

1.3 THE MESSAGE FLOW

The term message flow covers the task of when to send which particular network API control message in the context of time and circumstances. The term message flow addresses the micro as well as the macro message flow during a client/server session. The micro message flow covers the basic handshake between client and server, i.e. how the server reacts on a particular network API control message. The macro message flow deals with the overall network API control message flow to achieve a certain behavior of the g.NEEDaccess server, and thus solve a certain task using the server.

1.3.1 THE MICRO MESSAGE FLOW

In this section, the principal setup of a network API control message request and the server's reply is explained, including the handshake between them.

Control message request (from the client)

Once a socket connection between the server and a client is established, the client then initiates a control message request. The client must configure the 5 fundamental data fields of the XML string as follows, and send the message (including the meta-header) to the server over an already established connection:

- **Session ID/connection handle**
The session ID retrieved from the server, except the client requests a session ID at the beginning to establish a client/server session. If the client requests a session ID, the session ID must be 0.
- **Command**
The command field is set to a value from the list of valid commands.
- **Payload**
The payload may contain data providing additional parameters required by the corresponding command.
- **Status**
The status must be set to 1 for a command (i.e. request) that the client sends to g.NEEDaccess server (see section 3.1.2 *Status codes*)
- **Error**
The error must be set to 0 by the client (see section 3.1.1 *Error codes*).

The server confirms reception of the request with an acknowledge message, which the client must receive. The acknowledge message signals the client that the server has completely received the request only. It does not give any information about success or failure of actually processing the request. An acknowledge message for a specific request has the same package ID in its meta-header as the package ID in the meta-header of the original request. It has the acknowledge flag set and no XML string appended (i.e. a value of zero in the meta-header's size field).

1.3.2 CONTROL MESSAGE REPLY (FROM THE SERVER)

After the server has processed the request, it answers with a network API control message reply, except for the request to destroy the current session. The reply also consists of the meta-header and an XML string, which the g.NEEDaccess server configures as follows:

- **Session ID/connection handle**
The session ID of the concerned client. It is the same specified by the client when it sent the request.
- **Command**
The command is set to the command which has been processed according to the request of the client. Hence, it is the same specified by the client when it sent the corresponding request.
- **Payload**
Depending on the corresponding command, the server returns additional result data for the processed request in the payload field. This field will be empty for commands/requests that do not return additional result data.
- **Status**
If g.NEEDaccess server was able to process the request successfully, the status field is set to 2. On failure, the status field is set to 3
- **Error**
If the status field is set to 2, the error field is set to 0. Otherwise, the error field is set to a value from the list of defined error codes (see section 3.1.1 *Error codes*). The payload of an error message then holds a string with further information about that error. In the detailed network API reference, examples of server replies for specific requests are only given for successful requests.

The client must confirm reception of the reply with an acknowledge message as well. The package ID of the acknowledge message meta-header must be the same as the package ID in the meta-header of the received reply. The client must also set the acknowledge flag and send that meta-header back to the server, without an XML string appended (i.e. a value of zero in the meta-header's size field). The acknowledge message signals the server that the client has completely received the reply. No further handshake is required.

1.3.3 THE MACRO MESSAGE FLOW

To achieve a certain behavior of g.NEEDaccess server, one or more micro messages must be sent. Such a logic compound of multiple micro messages is called macro message.

This section lists the composition of provided macro messages common to all devices and gives an example of the typical message flow for a simple data acquisition task.

Concrete definition of macro messages

A macro message is simply a name or task that addresses a composition of one or multiple micro messages. The following table will introduce the common macro messages. The detailed description of each of those micro messages follows in section *Network API reference*. See also the descriptions of the corresponding C API functions in the g.NEEDaccess Client API documentation for details on the behavioral effect.

Macro message name	Micro messages (in correct ordering)	Corresponding C API function
Retrieve connected devices	Get a session ID from g.NEEDaccess server (GET_SESSION_ID) Get a session ID from g.NEEDaccess server (GET_SESSION_ID)	
	Get a list of supported devices connected to g . NEEDaccess server (GET_CONNECTED_DEVICES)	GDS_RESULT GDS_GetConnectedDevices (...)
	Disconnect from g.NEEDaccess server (DISCONNECT)	
Connect to g.NEEDaccess server	Get a session ID from g.NEEDaccess server (GET_SESSION_ID)	
	Setup a data streaming socket (SETUP_STREAMING)Setup data streaming socket (SETUP_STREAMING)	GDS_RESULT GDS_Connect (...)
	Open or create a data acquisition session (OPEN_DAQ_SESSION , OPEN_DAQ_SESSION_EXCLUSIVELY)	
Disconnect from g.NEEDaccess server	Close data acquisition session (CLOSE_DAQ_SESSION)	GDS_RESULT
	Disconnect from g.NEEDaccess server (DISCONNECT)	GDS_Disconnect (...)
Set configuration	Configure devices of a data acquisition session (SET_CONFIGURATION)	GDS_RESULT GDS_SetConfiguration (...)
Get configuration	Get configuration of devices of a data acquisition session (GET_CONFIGURATION)	GDS_RESULT GDS_GetConfiguration (...)
Become participant	Become participant of a data acquisition session (BECOME_PARTICIPANT)	GDS_RESULT GDS_BecomeParticipant (...)
Become creator	Become creator of a data acquisition session (BECOME_CREATOR)	GDS_RESULT GDS_BecomeCreator (...)
Start acquisition	Start data acquisition (START_ACQUISITION)Start data acquisition (START_ACQUISITION)	GDS_RESULT GDS_StartAcquisition (...)
Stop acquisition	Stop data acquisition (STOP_ACQUISITION)Stop data acquisition (STOP_ACQUISITION)	GDS_RESULT GDS_StopAcquisition (...)

Start streaming	Start streaming data (START_STREAMING)	GDS_RESULT GDS_StartStreaming(...)
Stop streaming	Stop streaming data (STOP_STREAMING)	GDS_RESULT GDS_StopStreaming(...)
Get data info	Get information about data organization (GET_DATA_INFO)	GDS_RESULT GDS_GetDataInfo(...)
Get data	Retrieve acquired data from the server (GET_DATA)	GDS_RESULT GDS_GetData(...)

The message flow of a typical DAQ session

Based on a regular DAQ session, we can derive following workflow, which will be described in the following section. The order in which the commands are sent to the server is crucial. First of all, a TCP/IP socket connection between the server and the client must be established before any control messages can be sent.



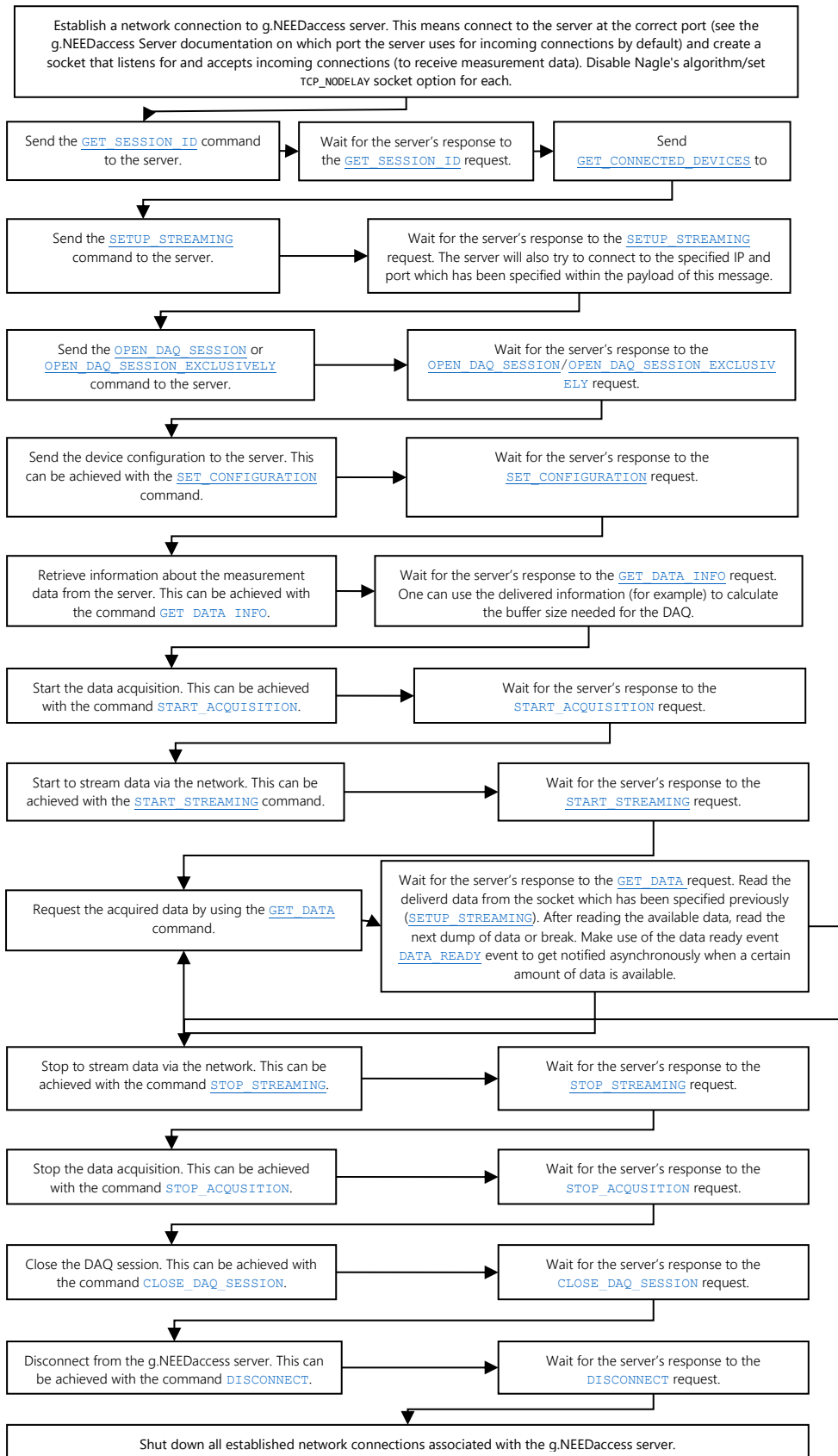
IMPORTANT

Disable Nagle's algorithm used by TCP after opening a socket but before connecting the socket or right after accepting an incoming connection at both client and server sides. Do so by setting and enabling the TCP_NODELAY socket option. Consult the API reference of your operating system or the socket library you are using for details on how to accomplish that.

At first, the session must be established by retrieving a session ID from the server. The user may be interested in the connected devices, so a list of supported devices physically connected to the server can be retrieved using the [GET_CONNECTED_DEVICES](#) command (this request is not required). The next step is to open a DAQ session with [OPEN_DAQ_SESSION](#) or [OPEN_DAQ_SESSION_EXCLUSIVELY](#) and configure the devices that belong to that DAQ session using [SET_CONFIGURATION](#). The easiest way to determine the required buffer size for data retrieval at the client side is a call to [GET_DATA_INFO](#) before data acquisition starts, but the call is also not required. [START_ACQUISITION](#) starts the process of data acquisition on the device and the server, but does not stream data to the client machine yet. Once the [START_STREAMING](#) command is sent successfully, the client can request data with [GET_DATA](#) (see later). [STOP_STREAMING](#) will stop data delivery to the client machine, while data acquisition on the device and the server remains running until [STOP_ACQUISITION](#) is issued. The division of the methods [START_ACQUISITION](#) and [START_STREAMING](#), as well as [STOP_STREAMING](#) and [STOP_ACQUISITION](#), was done to enable the server to deal with a multiple participants within a DAQ session. Only the session creator is allowed to control (start, stop, configure) data acquisition and the device, while simple participants are only allowed to "listen" to a data stream of an existing DAQ session (i.e. receive data) but cannot control data acquisition or the device. Acquired data is buffered at the server and can be transferred using the [GET_DATA](#) request for each creator or participant that has streaming enabled. Data is buffered for each of them separately so that everyone gets the whole data stream. To stop data acquisition, first call [STOP_STREAMING](#) for each participant and the creator, and then [STOP_ACQUISITION](#) for the creator. If no further interaction with the server is needed, the entire DAQ session can be closed with the disconnect request [CLOSE_DAQ_SESSION](#) followed by [DISCONNECT](#).

During all these interactions, the server may trigger asynchronous events anytime. It is up to the user to receive and react to these events. The most critical one is the heartbeat message, which **MUST** be handled (i.e. acknowledged), otherwise the server will consider the client dead and terminate the connection.

The following chart illustrates the message flow described above.



2 THE GDSNETWORKAPIDEMO PROJECT

GDSNetworkAPIDemo is a console application that illustrates the basic usage of the g.NEEDaccess network API in C++ programming language for Ubuntu Linux and Microsoft Windows operating systems. It also serves as a template for implementation of general g.NEEDaccess message handling and communication. Relevant source code is located in the *GDSNetworkAPIDemo* folder, which is installed as an example with the installer for MS Windows. See the g.NEEDaccess Client API documentation on where to find examples after installation of g.NEEDaccess Client API.

A running instance of g.NEEDaccess Server on a host computer running Microsoft Windows OS is required to run the example. If the example is executed on the same machine that runs the server, the loopback adapter address can be used for communication with the server (see the g.NEEDaccess Server documentation for details). The following section shows how to compile the example's source code with the GNU C++ Compiler `g++` on Ubuntu Linux operating systems and with Microsoft Visual C++ (part of Microsoft Visual Studio) on MS Windows operating systems.

2.1 COMPILING THE GDSNETWORKAPIDEMO PROJECT ON LINUX

To verify that your Linux distribution has the `g++` compiler installed, open a terminal window and execute the command:

```
g++ -v
```

If your system is set up correctly, this command will launch the compiler executable and print its version. If an error message is printed instead, make sure the compiler is installed and set up correctly. The `pthread` and `rt` libraries are required in addition to compile and run the example.

To compile the application, open a terminal window, go to the location where the *GDSNetworkAPIDemo* source code resides, and execute the command:

```
g++ *.cpp -pthread -lrt -o GDSNetworkAPIDemo
```

This command will generate an executable file named *GDSNetworkAPIDemo* located in the same directory as the source code.

2.2 COMPILING THE GDSNETWORKAPIDEMO PROJECT ON WINDOWS

Start Visual Studio and create a Visual C++ project from existing source code by selecting *File* → *New* → *Project From Existing Code* from the menu. Follow the wizard and add all existing source code files from the *GDSNetworkAPIDemo* folder to the project. Once you have created the project, you can compile it by selecting *Build Solution* from the *Build* menu. The network API itself supports any platform, including 32-bit systems. It only requires a machine that runs g.NEEDaccess server.

A folder named according to the selected platform and configuration will be created in the project folder, which the resulting executable will be built to.

2.2.1 RUNNING THE EXECUTABLE

Run the executable either from Visual Studio, by double-clicking it, or from the command line using the following command:

```
./GDSNetworkAPIDemo <HostIp> <HostPort> <LocalIp> <LocalPort>
```

Replace <HostIp> and <HostPort> with the IP address and port that g.NEEDaccess server at the server machine is listening to. See the server's documentation for the default port. Replace <LocalIp> and <LocalPort> with the IP address and an arbitrary free port of the local machine that runs the demo. Ensure that your firewall settings permit connections at the specified local address. For example:

```
./GDSNetworkAPIDemo 192.168.1.43 50223 192.168.1.30 50224
```

On startup, the application scans for supported devices connected to the server at the specified address (192.168.1.43:50223 in the examples above) and prompts you to choose one of them from the presented list. Afterwards, the application (running at 192.168.1.30:50224 in the examples above) prompts you to enter the desired sampling rate and duration of the acquisition. Be sure to enter a valid sampling rate that the selected device supports. The application will acquire data with the specified sampling rate for the specified duration from the selected device at the server. Acquired data will be saved in a binary file `data.bin` located in the same directory where the demo application resides.

2.2.2 READING THE RECORDED DATA FILE

The recorded data file contains acquired samples as 32-bit float values. Data is organized as a concatenation of scans. Each scan consists of exactly one sample for each acquired channel, starting with the first acquired channel. All samples of a scan are sampled at the same point in time. It can be thought of a two-dimensional matrix in the format channels x samples.

The following piece of code illustrates how to read in the recorded data file in MATLAB. The `filename` variable must be set to the name of the recorded data file beforehand. The `channels` variable must be set to the number of acquired channels that the file contains beforehand, including the possibly recorded trigger channel (or other available additional channels). The number of acquired channels can be found through the methods that set up configuration for the corresponding device type in the source code (`SetupXMLConfig.hpp/.cpp`). The data variable receives the recorded data from the file as a matrix in the format channels x samples.

```
filename = 'data.bin';
channels = 16;

fid = fopen(filename, 'rb');
data = fread(fid, [channels, inf], 'float32');
fclose(fid);
```

3 NETWORK API REFERENCE

In this section, the concrete network API messages are described in detail, together with the server's responses.



ATTENTION

If a message, no matter whether request, reply or event, requires or delivers content in the payload field, the content is given in non-escaped XML syntax in the detailed description of the message. To send a message with payload content, that XML statement must generally be converted to escaped XML syntax before sending it as payload content. When receiving a message, the payload content comes as escaped XML syntax in turn. It must be converted to non-escaped XML syntax to comply with the given descriptions. See section

Escaping the payload for details.

3.1 COMMON FUNCTIONALITY

The messages described in this section only apply to all supported device and source types.

3.1.1 ERROR CODES

The following table lists all possible values that may be returned as error code in the message's XML string from the server.

Value	Canonical name	Description
0	GDS_ERROR_SUCCESS	The operation completed successfully. No error has occurred.
1	GDS_ERROR_INVALID_PARAMETER	One of the specified parameters does not contain a valid value.
2	GDS_ERROR_INVALID_HANDLE	The specified connection handle is invalid.
3	GDS_ERROR_HANDLE_NOT_ASSOCIATED	The specified connection handle is not associated with any existing data acquisition session created or opened correct.
4	GDS_ERROR_HANDLE_ALREADY_ASSOCIATED	The specified connection handle is already associated with an existing data acquisition session.

5	GDS_ERROR_ACCESS_DENIED	An existing data acquisition session couldn't be opened because it has been opened exclusively by its creator, or the specified connection handle is not creator of its associated data acquisition session, or the data acquisition session associated with the specified connection handle already has a creator.
6	GDS_ERROR_INVALID_DEVICE_LIST	The specified list of devices contains no or invalid entries.
7	GDS_ERROR_INVALID_DEVICE	A device cannot be part of the same data acquisition session than the others due to incompatible device type or settings, for example, or the specified device is not part of the specified data acquisition session.
8	GDS_ERROR_OPEN_DEVICE	A device could not be opened.
9	GDS_ERROR_DEVICE_ALREADY_IN_USE	A device is already part of an existing data acquisition session with a different list of associated devices, or an attempt was made to open an already existing data acquisition session exclusively.
10	GDS_ERROR_INVALID_CONFIGURATION	A device could not be configured with the specified configuration, or is not configured at all.
11	GDS_ERROR_INCOMPATIBLE_CONFIGURATIONS	The device configurations are not compatible among themselves with respect to the associated data acquisition session.
12	GDS_ERROR_READ_CONFIGURATION	The configuration for a device could not be retrieved.
13	GDS_ERROR_INVALID_ACQUISITION_STATE	The operation is not allowed during the current acquisition state.
14	GDS_ERROR_INVALID_STREAMING_STATE	The operation is not allowed during the current streaming state.
15	GDS_ERROR_BUFFER_TOO_SMALL	The specified number of elements to read exceeds the specified destination buffer's size.

16	GDS_ERROR_ACQUISITION_BUFFER_OVERFLOW	An overflow occurred in the internal acquisition buffer of a data acquisition session. The server did not fetch data from the acquisition unit within a reasonable time. This indicates a problem with the server. The buffer will be reset automatically, and the client can continue data acquisition.
17	GDS_ERROR_TRANSMISSION_BUFFER_OVERFLOW	An overflow occurred in a client's streaming buffer on the server. The client did not fetch data from the server in a reasonable time. The buffer will be reset automatically, and the client can continue data acquisition.
18	GDS_ERROR_ACQUISITION_DIED	An internal error occurred during a data acquisition session on the server forcing data acquisition for that session to stop. Connections are kept alive, and the creator of the concerned session can start data acquisition again.
19	GDS_ERROR_SERIALIZATION	A message or command could not be sent to the server because its content couldn't be serialized on the client side or deserialized. This indicates an internal error on the client or server side.
20	GDS_ERROR_INVALID_ENDPOINT	The specified endpoint does not exist, or no server is running there.
21	GDS_ERROR_INVALID_REQUEST_STATUS	The status of the request is invalid.
22	GDS_ERROR_REQUEST_TIMED_OUT	The server did not reply to the sent request within a reasonable time.
0xFFFF FFFF	GDS_ERROR_UNKNOWN	An unknown error occurred.

3.1.2 STATUS CODES

The following table lists all possible values that may appear in the status field of the message's XML string. When a client issues a message to the server, it must set the proper status code value, depending on the type of the message.

Value	Canonical name	Description
0	NO_STATUS_CODE	The message is an asynchronous event message sent from the server to the client.
1	COMMAND	The message is a command request from the client to the server.
2	SUCCESS	The message is a reply to the client from the server, which succeeded in processing a previously issued request from the client.
3	FAILURE	The message is a reply to the client from the server, which failed in processing a previously issued request from the client.

3.1.3 CONTROL MESSAGES

The following table lists all provided control messages that are common to all devices, along with their command values that a client must set in the command field of the message's XML string.

Value	Canonical name	Description
0	NO_COMMAND	This command code is not used for any control message. It would reveal messages that didn't get assigned a command value.
2	GET_CONNECTED_DEVICES	Get a list of supported devices connected to g.NEEDaccess server.
3	OPEN_DAQ_SESSION	Open or create a data acquisition session.
4	OPEN_DAQ_SESSION_EXCLUSIVELY	Open or create a data acquisition session.
5	CLOSE_DAQ_SESSION	Close data acquisition session.
6	BECOME_CREATOR	Become creator of a data acquisition session.
7	GET_SESSION_ID	Get a session ID from g.NEEDaccess server.
8	BECOME_PARTICIPANT	Become participant of a data acquisition session.
9	START_ACQUISITION	Start data acquisition.
10	STOP_ACQUISITION	Stop data acquisition.
11	START_STREAMING	Start streaming data.
12	STOP_STREAMING	Stop streaming data.
13	GET_DATA_INFO	Get information about data organization.
14	GET_DATA	Retrieve acquired data from the server.
15	SET_CONFIGURATION	Configure devices of a data acquisition session.
16	GET_CONFIGURATION	Get configuration of devices of a data acquisition session.

22	DISCONNECT	Disconnect from g.NEEDaccess server.
30	SETUP_STREAMING	Setup data streaming socket.
31	DEVICE_SPECIFIC_OPERATION	Execute a device-specific operation.
33	SET_DATA_READY_EVENT_THRESHOLD	Set data ready event threshold.

A detailed description of these messages follows.

Get a session ID from g.NEEDaccess server (GET_SESSION_ID)

The server identifies and associates a connected client with a unique ID called session ID. Thus, the first thing a client must do after establishing the socket connection is to request such a session ID, by sending a control message with the value of the `GET_SESSION_ID` command in the command field. The status field must be set to the value of the `COMMAND` status, as for all control requests. The session ID field must hold a value of 0, identifying a new client that is not associated with any session ID yet. On success, the server replies with a message containing the client's new session ID. This session ID must be kept and specified in each control request that client issues to the server until disconnection.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>7</command_>
    <payload_></payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>7</command_>
    <payload_></payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Setup a data streaming socket (SETUP_STREAMING)

The server sets up an extra TCP/IP socket for each client. This additional socket is used to transfer acquired data only, to avoid blocking the server's control pipe. The command field must be set to the value of the `SETUP_STREAMING` command. The client must specify a local endpoint (IP address and port number) in the payload of the message to which the server should connect and transfer acquired data, in case data acquisition is started later. In the following example, the client tells the server to connect to the client's endpoint at IP 127.0.0.1 on port 51082 to stream data to this endpoint. Please choose the values of IP and port according to your needs.

Before this command is issued, the client must create a socket for the specified local client-side endpoint and start listening for incoming connections. Immediately after issuing this command, the server tries to establish a TCP/IP socket connection with the specified local client-side endpoint. The client must accept this incoming connection.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>30</command_>
    <payload_>
      → insert escaped payload message of the client request here
    </payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <first>127.0.0.1</first>
    <second>51082</second>
  </value>
</boost_serialization>
```

**NOTE**

This XML expression corresponds to a serialized C++ `std::pair<std::string, uint16_t>`.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>30</command_>
    <payload_></payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Get a list of supported devices connected to g.NEEDaccess server (GET_CONNECTED_DEVICES)

Gets a list of devices connected to g.NEEDaccess server.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>2</command_>
    <payload_>
      → insert escaped payload message of the client request here
    </payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>0</value>
</boost_serialization>
```



NOTE

The value field itself holds a single number. The value field is set to 1 to derive information about the devices in use. Otherwise, the value field is set to 0 to derive information about all devices connected to the server. This XML expression corresponds to a serialized C++ bool value.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>2</command_>
    <payload_>
      → insert escaped payload message of the server's reply here
    </payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <count>2</count>
    <item_version>0</item_version>
    <item class_id="1" tracking_level="0" version="0">
      <InUse>1</InUse>
      <ConnectedDevices class_id="2" tracking_level="0" version="0">
        <count>1</count>
        <item_version>0</item_version>
        <item class_id="3" tracking_level="0" version="0">
          <Name>UB-2012.05.24</Name>
          <DeviceType>1</DeviceType>
        </item>
      </ConnectedDevices>
    </item>
    <item>
      <InUse>0</InUse>
      <ConnectedDevices>
        <count>1</count>
        <item_version>0</item_version>
        <item>
          <Name>UB-0000.00.01</Name>
          <DeviceType>1</DeviceType>
        </item>
      </ConnectedDevices>
    </item>
  </value>
</boost_serialization>
```



NOTE

The payload contains a serialized C++ `std::vector<...>` of size `<count>`, where each `<item class_id="1" ...>` element of that vector whose `bool` value in the `<InUse>` element is set to 1 represents an existing data acquisition unit. The `<ConnectedDevices>` element is a serialized C++ `std::vector<...>` that contains an `<item class_id="3" ...>` element for each device that belongs to that data acquisition unit. For each device, the serial number is given in the `<Name>` element (a serialized C++ `std::string`) and the `<DeviceType>` as an integral number (a serialized C++ enumeration value). `<DeviceType>` may contain one of the following values:

Device Type	Canonical name	Description
0	DEVICE_TYPE_NOT_SUPPORTED	The device is of a type that is not supported.
1	DEVICE_TYPE_GUSBAMP	A g.USBamp device.
2	DEVICE_TYPE_GHIAMP	A g.Hlamp device.
3	DEVICE_TYPE_GNAUTILUS	A g.Nautilus device.

The outermost vector may contain one additional element whose `<InUse>` element is set to 0, containing a list of all devices that are connected to the server but not currently in use by any of the existing data acquisition units.

The `<item_version>` element is the serialization infrastructure of the vector class and always contains zero. It does not imply information about the version of the devices in the vector.

Open or create a data acquisition session (`OPEN_DAQ_SESSION`,
`OPEN_DAQ_SESSION_EXCLUSIVELY`)

Creates a new data acquisition (DAQ) session or opens an existing one with the specified devices. On success, data acquisition can be started.

A data acquisition session is identified by the list of device names that are part of the session. If the request is done for a specific list of devices the first time, a new data acquisition session is created for these devices, data will be acquired synchronously for all devices in that list, and the newly created data acquisition session will be associated with the specified session ID. In this case, the server's reply contains a serialized Boolean value of 1, stating that the session ID that the request has been issued for is the creator of this new data acquisition session.

If the request is done with the exact same list of devices that have already been opened by a previous `OPEN_DAQ_SESSION` request for a different client (i.e. for a different session ID), the session ID that the current request has been issued for is considered a participant of the already existing data acquisition session, and the server returns a serialized Boolean value of 0 in the payload.

The request will fail if the specified list of devices contains at least one device that is already in use by an existing data acquisition session and the specified list of devices is not exactly equal to the list of devices that was specified for that existing session on its creation.

The request will fail if no device was specified, or if the server could not open at least one device in the list.

The request will also fail if an existing data acquisition session was opened exclusively.

It is only possible to open more than one device in the same DAQ session if the devices support hardware synchronization.

See the g.NEEDaccess Client API documentation for details on the terms *creator* and *participant* of a data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>3</command_>
    <payload_>
      → insert escaped payload message of the client request here
    </payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

**NOTE**

A data acquisition session can be opened exclusively or not. To open the data acquisition session exclusively, use the `OPEN_DAQ_SESSION_EXCLUSIVELY` command value. In exclusive mode, only the creator can acquire data from the device, and no client can open the existing data acquisition session as a participant. Otherwise, use the `OPEN_DAQ_SESSION` command value.

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>UA-2007.04.13</item>
  </value>
</boost_serialization>
```

**NOTE**

The payload is a serialized C++ `std::vector<string>` containing all device names that should be associated with the data acquisition session. The `<count>` element must hold the number of devices specified, where each `<item>` element contains the name (i.e. serial number) of one device. Ordering matters. If the devices support hardware synchronization and more than one device is specified in the list, the first device in the list is considered the master device, while the others are considered slave devices in the specified order.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>3</command_>
    <payload_>
      → insert escaped payload message of the server's reply here
    </payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of server's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>1</value>
</boost_serialization>
```



NOTE

The server returns a serialized C++ bool value indicating whether the client with the specified session ID is creator (a value of 1) or participant (a value of 0) of the data acquisition session.

Close data acquisition session (CLOSE_DAQ_SESSION)

Dissociates the specified session ID from its associated data acquisition session. If the specified session ID was the creator of its associated data acquisition session, the whole data acquisition session will be destroyed, all associated participants get dissociated (as if the CLOSE_DAQ_SESSION command was sent for each of them as well), and the [SESSION_CREATOR_DIED](#) event is raised. Otherwise, only the specified participant's session ID gets dissociated; that is, the data acquisition session remains running for all remaining session IDs that are still associated with it and for which CLOSE_DAQ_SESSION has not been called previously.

If only participants are still associated with the data acquisition session (maybe because the creator has died unexpectedly or the creator has resigned from its creator role and became a simple participant), the server closes the data acquisition session when the last participant closes its data acquisition session.

The request will fail if the specified session ID is not associated with any existing data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>5</command_>
    <payload_></payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>5</command_>
    <payload_></payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Server event

The [SESSION_CREATOR_DIED](#) event is raised for each participant if the specified session ID belongs to a creator of an existing data acquisition session.

Disconnect from g.NEEDAccess server (DISCONNECT)

Disconnects the client with the specified session ID from the server. The server releases all resources it has used for maintaining the client and communication with it. The socket connection for control and data sockets for that client can be closed afterwards.

The [CLOSE_DAQ_SESSION](#) command must have been called previously for the specified session ID, unless it is a participant and the [CLOSE_DAQ_SESSION](#) command has already been called for the creator of the associated data acquisition session. In the latter case, only the `DISCONNECT` request must be sent.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>22</command_>
  <payload_></payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>22</command_>
    <payload_></payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Configure devices of a data acquisition session (SET_CONFIGURATION)

Sets the configuration for each device in the data acquisition session that was created with the specified session ID. The payload contains a list of pairs with one device-specific configuration entry for each device in the data acquisition session. See section 3.2 *Device specifics (general)* and the section for specifics of the corresponding device type for details on the device-specific representation of the configuration for each device type.

The request will fail if the specified session ID does not belong to the creator of the corresponding data acquisition session, or if the specified session ID is not associated with any existing data acquisition session.

The request will fail if the specified configuration list in the payload does not match the list that was specified for the same session ID on creation.

The request will fail if at least one device could not be configured with the specified configuration due to some invalid parameters, or invalid combinations of parameters in the corresponding configuration.

The request will fail if the device configurations are not compatible among themselves with respect to the associated data acquisition session.

The request will fail if data acquisition is currently running for the associated data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization:archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>15</command_>
    <payload_>
      → insert escaped payload message of the client request here
    </payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

**NOTE**

The payload is a serialized C++ `std::map<std::string, configuration>` with the serial number of the device in its keys and the corresponding device-specific representation of the configuration in its values. The list of devices must be in the same order as previously specified in [OPEN_DAQ_SESSION](#) or [OPEN_DAQ_SESSION_EXCLUSIVELY](#), i.e. the element at index i in the vector holds the configuration for the device's serial number specified at index i in the list of [OPEN_DAQ_SESSION](#) or [OPEN_DAQ_SESSION_EXCLUSIVELY](#).

See section 3.2 *Device specifics (general)* and the section for specifics of the corresponding device type for details on the device-specific representation of the configuration for each device type.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization:archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>15</command_>
    <payload_></payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Server event

The [CONFIGURATION_CHANGED](#) event is raised for all participants of the associated data acquisition session on success.

Get configuration of devices of a data acquisition session (GET_CONFIGURATION)

Gets the configuration for each device in the associated data acquisition session. The payload of the server's reply contains a list of pairs with one device-specific configuration entry for each device in the data acquisition session.

See section 3.2 *Device specifics (general)* and the section for specifics of the corresponding device type for details on the device-specific representation of the configuration for each device type.

The request will fail if the specified session ID is not associated with any existing data acquisition session.

The request will fail if the server could not retrieve the configuration for at least one device in the associated data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>16</command_>
    <payload_></payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>16</command_>
    <payload_>
      → insert escaped payload message of the server's reply here
    </payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

**NOTE**

The payload is a serialized C++ `std::map<std::string, configuration>` with the serial number of the device in its keys and the corresponding device-specific representation of the configuration in its values. The list of devices will be in the same order as previously specified in [OPEN_DAQ_SESSION](#) or [OPEN_DAQ_SESSION_EXCLUSIVELY](#), i.e. the element at index i in the vector holds the configuration for the device's serial number specified at index i in the list of [OPEN_DAQ_SESSION](#) or [OPEN_DAQ_SESSION_EXCLUSIVELY](#).

See section 3.2 *Device specifics (general)* and the section for specifics of the corresponding device type for details on the device-specific representation of the configuration for each device type.

Become participant of a data acquisition session (BECOME_PARTICIPANT)

Downgrades the creator of its associated data acquisition session to a simple participant. This request is only allowed for session IDs that represent creators of a data acquisition session.

The request will fail if the specified session ID is not a creator of its associated data acquisition session, or if the specified session ID is not associated with any existing data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>8</command_>
    <payload_></payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>8</command_>
    <payload_></payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Server event

The [SESSION_CREATOR_DIED](#) event is raised for all participants of the associated data acquisition session on success.

Become creator of a data acquisition session (BECOME_CREATOR)

Elevates a participant to the creator of its associated data acquisition session if no creator currently exists for that data acquisition session. This request is only allowed for session IDs that represent participants of a data acquisition session that currently has no creator (maybe because the creator has died unexpectedly, or the creator has resigned from its creator role and became a simple participant).

The request will fail if the specified session ID is not a participant of its associated data acquisition session, or if the specified session ID is not associated with any existing data acquisition session.

The request will fail if the data acquisition session associated with the specified session ID does already have an active creator.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>6</command_>
  <payload_></payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>6</command_>
  <payload_></payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server event

The [NEW_SESSION_CREATOR](#) event is raised for all participants of the associated data acquisition session on success.

Start data acquisition (START_ACQUISITION)

Starts data acquisition on all devices in the data acquisition session that was created with the specified session ID. Issue a [START_STREAMING](#) request afterwards to be able to retrieve data with [GET_DATA](#). See section

The message flow of a typical DAQ session for details on [START_ACQUISITION](#), [START_STREAMING](#), and [GET_DATA](#).

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if it is not creator of its associated data acquisition session.

The request will fail if the devices in the associated data acquisition session have not been configured properly yet.

The request will fail if data acquisition is already running for the associated data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>9</command_>
  <payload_></payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>9</command_>
  <payload_></payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server event

The [DATA_ACQUISITION_STARTED](#) event is raised for all participants of the associated data acquisition session on success.

The [DATA_ACQUISITION_ERROR](#) event may be raised whenever a server-side error occurs during data acquisition.

Stop data acquisition (STOP_ACQUISITION)

Stops data acquisition on all devices of the data acquisition session created with the specified session ID. Issue a [STOP_STREAMING](#) request before. See section

The message flow of a typical DAQ session for details on [STOP_ACQUISITION](#), [STOP_STREAMING](#), and [GET_DATA](#).

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if it is not a creator of its associated data acquisition session.

The request will fail if data acquisition is not running for the associated data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>10</command_>
  <payload_></payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>10</command_>
  <payload_></payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server event

The [DATA_ACQUISITION_STOPPED](#) event is raised for all participants in the associated data acquisition session on success.

Start streaming data (START_STREAMING)

Enables data streaming for the specified session ID. [GET_DATA](#) can be used afterwards to retrieve acquired data. Data acquisition must have been started previously by the creator with [START_ACQUISITION](#) for that session. See section

The message flow of a typical DAQ session for details on [START_ACQUISITION](#), [START_STREAMING](#), and [GET_DATA](#).

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if data acquisition is not running for the associated data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>11</command_>
  <payload_></payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>11</command_>
  <payload_></payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Stop streaming data (STOP_STREAMING)

Disables data streaming for the specified session ID. Data acquisition remains running for session IDs that still have streaming enabled. See section

The message flow of a typical DAQ session for details on [STOP_ACQUISITION](#), [STOP_STREAMING](#), and [GET_DATA](#).

The request will fail if the specified session ID is not associated with any existing data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>12</command_>
  <payload_></payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>12</command_>
  <payload_></payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Get information about data organization (GET_DATA_INFO)

Returns information about organization of the data retrieved with [GET_DATA](#) for the specified session ID and the specified number of scans.

A sample or element is the 32-bit floating point value of exactly one channel. A scan consists of exactly one sample for each acquired channel. The total number of acquired channels across all devices is the sum over all elements of the returned channels per device vector, which is the number of elements in a single scan. The required size of the buffer needed to hold the specified or returned number of scans (in bytes) is *buffer size in samples * sizeof(float)*.

The request will fail if the specified session ID is not associated with any existing data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>13</command_>
    <payload_>
      → insert escaped payload message of the client request here
    </payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
  <boost_serialization signature="serialization::archive" version="19">
    <value>1</value>
  </boost_serialization>
</boost_serialization>
```

**NOTE**

The payload content of the request contains a serialized C++ `size_t` value that specifies the number of scans to retrieve the buffer size (and other information) for, or zero to retrieve information about data organization for the number of currently available scans instead. A value of 1 retrieves information for one single scan.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>13</command_>
    <payload_>
      → insert escaped payload message of the server's reply here
    </payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of server's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <number_of_scans_>1</number_of_scans_>
    <channels_per_device_>
      <count>1</count>
      <item_version>0</item_version>
      <item>16</item>
    </channels_per_device_>
    <buffer_size_in_samples_>16</buffer_size_in_samples_>
  </value>
</boost_serialization>
```

**NOTE**

A serialized C++ structure is returned in the reply's payload content, holding the following parameters. The `<number_of_scans_>` element contains the number of scans for which the remaining parameters were calculated for as serialized C++ `size_t`. It is the same value that was passed in the client's request. If a value of 0 was specified in the client's request, the number of currently available and unclaimed scans at the server is returned in this element.

The `<channels_per_device_>` element contains a serialized C++ `std::vector<size_t>` with one `<item>` element for each device that is part of the acquisition. Its value is the number of channels acquired for the corresponding device, including synchronously sampled trigger channels and other additional channels. The element at index *i* corresponds to the device at index *i* in the device names list that has been specified on creation of the data acquisition session. The value in the `<count>` element equals the number of devices part of the data acquisition session, i.e. the number of `<item>` elements.

The `<buffer_size_in_samples_>` contains the number of samples that will make up the returned number of scans as serialized C++ `size_t`. This is the number of elements a buffer must contain to be able to hold that amount of scans.

Retrieve acquired data from the server (GET_DATA)

Causes the server to send the requested number of scans to the socket established for data transmission for the specified session ID with [SETUP_STREAMING](#). The client must receive data at that socket (see section 1.2.5 *Data transmission* for details on data transmission packages). The reply will be sent only after data transmission has completed or when an error occurs.

The server may transfer fewer scans than requested but no more scans than requested, depending on how many scans are currently available for transmission at the server, and returns the number of actually read scans in the reply. It will not block until the requested number of scans is available. Use the [DATA_READY](#) event together with the [SET_DATA_READY_EVENT_THRESHOLD](#) command to emulate a blocking behavior. The request also contains information about how much memory is available for storing data on the client. The provided buffer size must be large enough to handle the requested number of scans.

Data is retrieved in microvolts as concatenated scans. A scan consists of exactly one sample for each acquired channel. A sample or element is the 32-bit floating point value of exactly one channel in microvolts. The total number of acquired channels across all devices is the sum over all elements of the returned channels per device vector, which is the number of elements in a single scan. The required size of the buffer in bytes needed to hold the specified or returned number of scans is *buffer size in samples * sizeof(float)*.

The request will fail if the specified session ID is not associated with any existing data acquisition session.

The request will fail if acquisition or streaming has not already been started for the specified session ID.

The request will fail if the specified number of scans to retrieve is greater than zero, and the specified destination buffer size is not large enough to receive the specified number of scans.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization:archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>14</command_>
    <payload_>
      → insert escaped payload message of the client request here
    </payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <number_of_scans_>1600</number_of_scans_>
  <channels_per_device_>
    <count>0</count>
    <item_version>0</item_version>
  </channels_per_device_>
  <buffer_size_in_samples_>25600</buffer_size_in_samples_>
</value>
</boost_serialization>
```



NOTE

The payload content of the request contains a serialized C++ structure, holding the following parameters. The `<number_of_scans_>` element should be set to the maximum number of scans to transfer as serialized C++ `size_t`. If a value of 0 is specified, either all currently available and unclaimed scans at the server are transferred, or as many scans as the specified destination buffer size can contain are transferred, whichever is smaller.

The `<channels_per_device_>` element is not used for this request and may contain an empty serialized C++ `std::vector<size_t>`.

The `<buffer_size_in_samples_>` should be set to the maximum number of samples that the buffer that receives written data can contain.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>14</command_>
    <payload_>
      → insert escaped payload message of the server's replay here
    </payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of server's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>1312</value>
</boost_serialization>
```

**NOTE**

The payload content of the server's reply is a serialized C++ `size_t` with the number of actually transferred scans. In this example, the available number of scans is smaller than the requested number of scans, which is valid behavior

Set data ready event threshold (SET_DATA_READY_EVENT_THRESHOLD)

Enables or disables triggering of the [DATA_READY](#) event whenever the specified amount of scans is available at the server for the specified session ID. To disable the [DATA_READY](#) event for the specified session ID, set the threshold value to 0.

The request will fail if the specified session ID is not associated with any existing data acquisition session.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>33</command_>
    <payload_>
      → insert escaped payload message of the client request here
    </payload_>
    <status_>1</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of server's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>256</value>
</boost_serialization>
```

**NOTE**

The payload content is a serialized C++ `uint64_t` value specifying the number of scans that must be acquired by and available at the server, at least to trigger the [DATA_READY](#) event.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>14</command_>
    <payload_>
      → insert escaped payload message of the server's reply here
    </payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Server event

The [DATA_READY](#) event is raised for the specified session ID whenever a number of scans greater than zero specified in the request is available for transmission at the server.

Execute a device-specific operation (DEVICE_SPECIFIC_OPERATION)

This request is supposed to handle all operations that are not common to all devices types. The payload specifies all information and parameters required to execute a device specific operation (e.g. the target device, the operation itself, its parameters...). The device-specific operations a particular device type provides and its parameters are described in an extra section for each device type. See section 3.2.2 *Device-specific control messages* for details on device-specific operations.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or the specified target device is not part of the associated data acquisition session.

The request may fail with unknown error if the device-specific operation fails.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>3</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

Each device-specific operation is built up of three major elements:

- The name (i.e. serial number) of the device that the request targets.
- The device-specific command value. In this example, the [GET_SCALING](#) request for a g.USBamp device is used for illustration.
- A custom ID value is intended for device-specific operations that register a device-specific event handler at the server. For common device-specific operations, this value can be set to zero, unless the particular operation indicates otherwise.
- A vector that stores the parameters. The vector is a serialized C++ `std::vector<std::string>`, where each string entry is a serialized parameter. Its content depends on the device-specific operation used.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>31</command_>
    <payload_>
      → insert escaped payload message of the server's reply here
    </payload_>
    <status_>2</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

3.1.4 ASYNCHRONOUS EVENT MESSAGES

The following table lists all asynchronous event messages that the server can send to a client any time. These events are sent to the creator and all participants of a session. Each of them must receive the events. The value in the session ID field of the message's XML string will be set to the session ID that the event targets. The command value in the command field of the message's XML string identifies the event and is given in the table as well.

Command Value	Canonical name	Description
21	<u>HEARTBEAT_MISSING_SHUTDOWN</u>	The event that is raised when the server does not receive an acknowledge message for an issued heartbeat message.
23	<u>CONFIGURATION_CHANGED</u>	The event that is raised when the creator of a data acquisition session has changed the configuration of the associated devices.
24	<u>DATA_ACQUISITION_STARTED</u>	The event that is raised when the creator of a data acquisition session has started acquisition.
25	<u>DATA_ACQUISITION_STOPPED</u>	The event that is raised when the creator of a data acquisition session has stopped acquisition.
26	<u>DATA_ACQUISITION_ERROR</u>	The event that is raised when an error occurs during data acquisition.
27	<u>SESSION_CREATOR_DIED</u>	The event that is raised when the creator of a data acquisition session has died unexpectedly, or when a <u>BECOME_PARTICIPANT</u> request was performed successfully for the associated data acquisition session.
28	<u>NEW_SESSION_CREATOR</u>	The event that is raised when a data acquisition session gets a new creator (i.e. a <u>BECOME_CREATOR</u> request was performed successfully for the associated data acquisition session).
29	<u>SERVER_DIED</u>	The event that is raised when the server was forced to shut down unexpectedly before all open connections or data acquisition sessions were closed.
32	<u>DEVICE_SPECIFIC_CALLBACK</u>	The server may raise this event on behalf of a particular device type. See the documentation of the device-specific asynchronous event messages that a particular device type may trigger.

34	<u>DATA_READY</u>	The event that is raised whenever the amount of scans specified in a previous <u>SET_DATA_READY_EVENT_THRESHOLD</u> request is available for transmission at the server for the specified session ID.
----	-----------------------------------	---

A detailed description of these messages follows.

Heartbeat missing event (HEARTBEAT_MISSING_SHUTDOWN)

The event that is raised when the server does not receive an acknowledge message for an issued heartbeat message. In this case, the server considers the client dead and shuts down the connection. It is raised for the dead client only if the client is still able to receive the event. If the affected client is a creator of a data acquisition session, the server will also raise the [SESSION_CREATOR_DIED](#) event for all participants of the associated data acquisition session, which will remain running.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>32</command_>
    <payload_></payload_>
    <status_>0</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Configuration changed event (CONFIGURATION_CHANGED)

The event that is raised when the creator of a data acquisition session has changed the configuration of the associated devices. It will be raised for all participants of the affected data acquisition session.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>23</command_>
    <payload_></payload_>
    <status_>0</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Data acquisition started event (DATA_ACQUISITION_STARTED)

The event that is raised when the creator of a data acquisition session has started acquisition. It will be raised for all participants in the affected data acquisition session.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>24</command_>
    <payload_></payload_>
    <status_>0</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Data acquisition stopped event (DATA_ACQUISITION_STOPPED)

The event that is raised when the creator of a data acquisition session has stopped acquisition. It will be raised for all participants of the affected data acquisition session.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>25</command_>
    <payload_></payload_>
    <status_>0</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Data acquisition error event (DATA_ACQUISITION_ERROR)

The event that is raised when an error occurs during data acquisition. Depending on the type of error, it will be raised for affected client(s) in the affected data acquisition session.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>26</command_>
    <payload_>
      → insert escaped payload message of the server's event message here
    </payload_>
    <status_>0</status_>
    <error_>17</error_>
  </value>
</boost_serialization>
```

The payload of server's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>Overflow of the connection's transmission buffer.</value>
</boost_serialization>
```

**NOTE**

The payload content of the server's event is a serialized C++ `std::string` containing the error message in the `<value>` element in escaped XML syntax. In this example, the event represents the `GDS_ERROR_TRANSMISSION_BUFFER_OVERFLOW` error.

Session creator died event (SESSION_CREATOR_DIED)

The event that is raised when the creator of a data acquisition session has died unexpectedly, or when a [BECOME PARTICIPANT](#) request was performed successfully for the associated data acquisition session. It will be raised for all participants of the affected data acquisition session.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>27</command_>
    <payload_></payload_>
    <status_>0</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

New session creator event (NEW_SESSION_CREATOR)

The event that is raised when a data acquisition session gets a new creator (i.e. a [BECOME_CREATOR](#) request was performed successfully for the associated data acquisition session). It will be raised for all remaining participants in the affected data acquisition session.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>0</session_id_>
  <command_>28</command_>
  <payload_></payload_>
  <status_>0</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

Server died event (SERVER_DIED)

The event that is raised when the server was forced to shut down unexpectedly before all open connections or data acquisition sessions were closed. It will be raised for all clients that still have an open connection with the server.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>29</command_>
    <payload_></payload_>
    <status_>0</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Data ready event (DATA_READY)

The event that is raised whenever the number of scans specified in a previous [SET DATA READY EVENT THRESHOLD](#) request is available for transmission at the server for the specified session ID. It will be raised for affected clients, in the context of their data acquisition session, that have specified a threshold value greater than zero.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>34</command_>
    <payload_></payload_>
    <status_>0</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

Device-specific callback event (DEVICE_SPECIFIC_CALLBACK)

This event is supposed to be raised for any callback events that are not common to all device types. The payload will contain all information and parameters required to handle the callback (e.g. the source device that the callback was triggered for, the command value of the callback, or its parameters). The device-specific callbacks that a particular device type provides, and their parameters, are described in an extra section for each device type. See section 3.2.3 *Device-specific event messages* for details on device-specific callbacks.

The session ID will be set to the session ID of the client it targets.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <session_id_>0</session_id_>
    <command_>32</command_>
    <payload_>
      → insert escaped payload message of the server's event message here
    </payload_>
    <status_>0</status_>
    <error_>0</error_>
  </value>
</boost_serialization>
```

The payload of the message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <target_>NB-2014.01.01</target_>
    <command_>13</command_>
    <customId_>1</customId_>
    <parameters_ class_id="1" tracking_level="0" version="0">
    </parameters_>
  </value>
</boost_serialization>
```



NOTE

Each device-specific callback is built up of three major elements:

- The serial number of the source device.
- The device-specific callback's command value. In this example, the [IMPEDANCE MEASUREMENT CALLBACK](#) event for a g.Nautilus device is used for illustration.
- The custom value that the client provided with the device-specific operation that registered the callback to identify the source of the callback.
- A vector that stores the callback's parameters. The vector is a serialized C++ `std::vector<std::string>`, where each string entry is a serialized parameter.

3.2 DEVICE SPECIFICS (GENERAL)

While the messages described in the *Common functionality* section apply to all devices, a particular type of device may provide additional messages that only devices of that particular type support. These additional messages are called device specifics.

3.2.1 THE DEVICE CONFIGURATION

A special device-specific representative is the device configuration. It can be set and read with the common [SET CONFIGURATION](#) control message and the common [GET CONFIGURATION](#) control message request. But the content of the payload field of these messages differs for each device type.

3.2.2 DEVICE-SPECIFIC CONTROL MESSAGES

All other device-specific control messages follow a special encapsulation scheme. Each device-specific message is sent to the server with the common [DEVICE SPECIFIC OPERATION](#) control message.

The actual device-specific message is then contained in the [DEVICE SPECIFIC OPERATION](#) control message's payload as escaped XML syntax. Like the XML string of a common control message, the actual device-specific message is an XML serialized C++ class (including encapsulation tags). The following example illustrates the principal layout of a device-specific message, as it would be contained in the [DEVICE SPECIFIC OPERATION](#) control message's payload field before converting it to escaped XML syntax (the `VALUE` wildcard is to be replaced by the actual value).

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <target_>VALUE</target_>
    <command_>VALUE</command_>
    <customId_>VALUE</customId_>
    <parameters_ class_id="1" tracking_level="0" version="0">
      <count>VALUE</count>
      <item_version>0</item_version>
      <item>
        VALUE
      </item>
    </parameters_>
  </value>
</boost_serialization>
```

One `<item>VALUE</item>` entry for each required parameter. If a device-specific command does not require any parameter, the value within the `<count>` tag is set to 0 and no `<item>VALUE</item>` tags are contained here.

The fundamental data fields of this message are:

XML tag	Equivalent C++ data type	Description
<code><target_>VALUE</target_></code>	<code>std::string</code>	The serial number of the device that the device-specific operation targets.
<code><command_>VALUE</command_></code>	<code>int32_t</code>	The value identifying the device-specific operation's command. Each device type has its own numbering. Thus, the same value may represent a completely different command among different device types.
<code><customId_>VALUE</customId_></code>	<code>size_t</code>	This value is intended for device-specific operations that register a device-specific event handler at the server. The client that issues the device-specific operation assigns one unique value for the event handler to register in the <code><customId_></code> tag. When the server raises the registered event, the event will deliver the assigned value in its <code><customId_></code> tag as well, which enables the client handler to identify the source of the event. For common device-specific operations, this value can be set to zero, unless the particular operation indicates otherwise.
<pre> <parameters_ class_id="1" tracking_level="0" version="0"> <count>VALUE</count> <item_version>0</item_version> <item>VALUE</item> </parameters_> </pre>	<code>std::vector<std::string></code>	A list of parameters passed to the device-specific operation. The value within the <code><count></code> tag specifies the number of parameters. For each parameter, an <code><item></code> tag must be specified containing the parameter's value as an escaped XML string. If the operation does not require any parameters, the <code><count></code> tag is set to 0, and no <code><item></code> tags must be specified. However, the <code><count></code> tag and the <code><item_version></code> tag must always be specified.

On success, the reply from the server to such a request looks similar. It uses the same encapsulation scheme with the same XML syntax described above. The result or return values of

the device-specific operation are returned in the `parameters_` list, with one entry for each return value, or no entry if no return value exists.

If an error happens during processing of the device-specific operation, the reply to the [DEVICE_SPECIFIC_OPERATION](#) control message will indicate this error in the status field, and the payload will contain the error message instead. Device-specific operations may fail with an unknown error unless device-specific error codes are defined.

3.2.3 DEVICE-SPECIFIC EVENT MESSAGES

Device-specific callbacks, if provided for a particular device type, also follow an encapsulation scheme. The server issues a device-specific callback using the asynchronous [DEVICE_SPECIFIC_CALLBACK](#) event message.

The actual device-specific callback is then contained in the [DEVICE_SPECIFIC_CALLBACK](#) event message's payload as escaped XML syntax. It follows exactly the same syntax described above in the Device-specific control messages section, with the following differences in interpretation of the fields:

- The `<target_>` tag contains the serial number of the device that the callback was triggered for.
- The `<command_>` tag contains the command value identifying the device-specific callback. Each device type has its own numbering. Thus, the same value may represent a completely different callback among different device types.
- The `<customId_>` tag is intended to deliver the callback identifier that the client provided with the device-specific operation that registered the callback.
- The `<parameters_>` tag delivers the callback's parameters.

The client must not send back a reply to such a callback unless stated otherwise for a particular device-specific callback. However, the client has to acknowledge reception.

3.3 G.USBAMP SPECIFICS

The messages described in this section only apply to g.USBamp devices.

3.3.1 INTRODUCTION TO THE G.USBAMP CONFIGURATION

g.USBamp devices can be configured with the commonly available [SET_CONFIGURATION](#) control message. In turn, the currently set configuration can be read with the commonly available [GET_CONFIGURATION](#) control message request.

The payload of the [SET_CONFIGURATION](#) control messages, as well as the server's reply to a [GET_CONFIGURATION](#) request, contains the escaped XML representation of the g.USBamp configuration (also referred to as *g.USBamp configuration list*), which is described below. The g.NEEDaccess Client API documentation and the g.USBamp instructions for use provide detailed descriptions about the particular configuration parameters and their effects, as well as valid values.



ATTENTION

The description of the payload content (i.e. the g.USBamp configuration list) of the message is given as non-escaped XML statement here. When sending a message, that XML statement must generally be converted to escaped XML syntax first before setting it as payload. When receiving a message, the escaped XML syntax in the payload field must be converted to non-escaped XML statement.

3.3.2 THE g.USBAMP DEVICE CONFIGURATION

The g.USBamp configuration list is the key element in this message. The configuration of a g.USBamp device is built upon various elements:

Simple entries:

Element	Corresponding C++ data type	Description																										
sample_rate_	unsigned int	The sample rate of the analog inputs in hertz. Use GET SUPPORTED SAMPLING RATES for a list of valid values. Attention: Eight-channel g.USBamp devices support the following sample rates only: 128, 256, 512.																										
Number_of_scans	Size_t	The number of scans to retrieve from the device at once, or zero to determine the recommended number of scans for the specified sampling rate automatically. This value depends on the value of <code>sample_rate_</code>. The following table lists the recommended values of <code>number_of_scans_</code> for all possible sample rates. <table><tr><th>Sampling Rate [Hz]</th><th>Recommended value for number_of_scans_</th></tr><tr><td>32</td><td>1</td></tr><tr><td>64</td><td>2</td></tr><tr><td>128</td><td>4</td></tr><tr><td>256</td><td>8</td></tr><tr><td>512</td><td>16</td></tr><tr><td>600</td><td>32</td></tr><tr><td>1200</td><td>64</td></tr><tr><td>2400</td><td>128</td></tr><tr><td>4800</td><td>256</td></tr><tr><td>9600</td><td>512</td></tr><tr><td>19200</td><td>512</td></tr><tr><td>38400</td><td>512</td></tr></table>	Sampling Rate [Hz]	Recommended value for number_of_scans_	32	1	64	2	128	4	256	8	512	16	600	32	1200	64	2400	128	4800	256	9600	512	19200	512	38400	512
Sampling Rate [Hz]	Recommended value for number_of_scans_																											
32	1																											
64	2																											
128	4																											
256	8																											
512	16																											
600	32																											
1200	64																											
2400	128																											
4800	256																											
9600	512																											
19200	512																											
38400	512																											
slave_	bool	Indicates whether the device is operated in slave mode. If slave mode is disabled, the device is operated in master mode. If the device is specified as the first device in the list of devices to open, it must be set to master mode. Otherwise, the device must be operated in slave mode.																										
short_cut_	bool	Indicates whether the short cut socket is enabled. If short cut is enabled, a HIGH level on the SC input socket of the																										

amplifier disconnects the electrodes from the amplifier input stage as long as the HIGH level is applied. If short cut is disabled, the level on the SC input socket of the amplifier is ignored, and the amplifier always delivers the currently measured values on its input channels.

The operation mode of the device. The following table lists valid values:

device_mode_ enum

Value	Canonical name	Description
0	MODE_NORMAL	Measures data from the 16 synchronously sampled analog input channels.
2	MODE_CALIBRATE	Enables the internal signal generator and applies a test signal with the parameters specified in AnalogOut to all 16 synchronously sampled analog input channels.
3	MODE_COUNTER	Like MODE_NORMAL, but channel 16 will deliver a sample counter (with overrun at 1e6), if selected for acquisition.

Indicates whether the synchronously sampled digital trigger input channels should be included in data acquisition in an additional logical channel that is appended to the synchronously sampled analog input channels. If enabled, the values of the digital trigger input channels come encoded in a single value as an additional channel appended to the analog channels.

scan_dio_ bool

g.USBamp version 2.0 has just one trigger line, so the values of the trigger channel can be 0 (LOW) or 250,000 (HIGH).

g.USBamp version 3.0 has 8 trigger lines coming bit-encoded as UINT8 value on the trigger channel. If all of the 8 trigger lines are HIGH, the value of the channel equals 255.0. If e.g. trigger line 0 to 3 are HIGH, the value of the channel equals 15.0.

The *AnalogOut* entry to configure the internal signal generator, if enabled, with the following elements:

Element	Corresponding C++ data type	Description																		
shape_	enum	The wave shape of the signal generator. Valid values are:																		
		<table><thead><tr><th>Value</th><th>Canonical name</th><th>Description</th></tr></thead><tbody><tr><td>0</td><td>WAVESHAPE_SQUARE</td><td>A square wave.</td></tr><tr><td>1</td><td>WAVESHAPE_SAWTOOTH</td><td>A saw tooth.</td></tr><tr><td>2</td><td>WAVESHAPE_SINE</td><td>A sine wave.</td></tr><tr><td>3</td><td>WAVESHAPE_DRL</td><td>Driven Right Leg (DRL).</td></tr><tr><td>4</td><td>WAVESHAPE_NOISE</td><td>Noise.</td></tr></tbody></table>	Value	Canonical name	Description	0	WAVESHAPE_SQUARE	A square wave.	1	WAVESHAPE_SAWTOOTH	A saw tooth.	2	WAVESHAPE_SINE	A sine wave.	3	WAVESHAPE_DRL	Driven Right Leg (DRL).	4	WAVESHAPE_NOISE	Noise.
		Value	Canonical name	Description																
		0	WAVESHAPE_SQUARE	A square wave.																
		1	WAVESHAPE_SAWTOOTH	A saw tooth.																
		2	WAVESHAPE_SINE	A sine wave.																
3	WAVESHAPE_DRL	Driven Right Leg (DRL).																		
4	WAVESHAPE_NOISE	Noise.																		
Frenquency_	size_t	The frequency of the signal generator in hertz. Valid values range from 1 Hz to 100 Hz.																		
amplitude_	float	The amplitude of the signal generator in mV. Valid values range from -250 mV to +250 mV.																		
offset_	float	The offset of the signal generator in mV. Valid values range from -200 mV to +200 mV.																		

- The `common_ground_` entry to configure the common ground settings. Common ground is a serialized C++ `std::vector<bool>`. The vector always has a size of 4, representing the four groups (A, B, C, and D). Each element in the list indicates whether the ground socket of the corresponding group (A, B, C, or D) should be connected to common ground. The common ground setting for group A (channels 1 to 4) is represented by the element at index zero, group B (channels 5 to 8) at index 1, group C (channels 9 to 12) at index 2, and group D (channels 13 to 16) at index 3.

A value of 0 of an element in the list indicates that the ground socket of the corresponding group is not connected to the common ground.

A value of 1 indicates that the ground socket of the corresponding group is connected to the common ground.
- The `common_reference_` entry to configure the common reference settings. Common reference is a serialized C++ `std::vector<bool>`. The vector always has a size of 4, representing the four groups (A, B, C, and D). The common reference is set for each of the four groups A, B, C, and D. Each element in the list indicates whether the reference socket of the corresponding group (A, B, C, or D) should be connected to the common reference. The common reference setting for group A (channels 1 to 4) is represented by the element at index zero, group B (channels 5 to 8) at index 1, group C (channels 9 to 12) at index 2, and group D (channels 13 to 16) at index 3.

A value of 0 of an element in the array indicates that the reference socket of the corresponding group is not connected to the common reference.

A value of 1 indicates that the reference socket of the corresponding group is connected to the common reference.

- The `bandpass_` entry for the bandpass filter setting for each available synchronously sampled analog input channel. This entry is a serialized C++ `std::vector<int>`, where an entry at index *i* contains the zero-based index of the bandpass filter from the list of all available bandpass filters to use for channel *i* (the list of all available bandpass filters for a device can be retrieved with the [GET_BANDPASS_FILTERS](#) request). A value of -1 indicates that no bandpass filter should be used for the corresponding channel. The vector must always contain filter settings for all available synchronously sampled analog input channels.
- The `notch_` entry for the notch filter setting of each available synchronously sampled analog input channel. This entry is a serialized C++ `std::vector<int>`, where an entry at index *i* contains the zero-based index of the notch filter from the list of all available notch filters to use for channel *i* (the list of all available notch filters for a device can be retrieved with the [GET_NOTCH_FILTERS](#) request). A value of -1 indicates that no notch filter should be used for the corresponding channel. The vector must always contain filter settings for all available synchronously sampled analog input channels.
- The `bipolar_` entry for the bipolar derivation setting of each available synchronously sampled analog input channel. This entry is a serialized C++ `std::vector<unsigned int>` where an entry at index *i* contains the bipolar derivation setting for channel *i*. Set this value to zero if unipolar derivation should be used with ground and reference electrodes that are connected to the ground and reference socket of the group that the corresponding channel belongs to. Otherwise, if bipolar derivation should be performed for the corresponding channel, set the value to the one-based channel number whose signal should be subtracted from the measured signal of the corresponding channel.
- The `analog_in_` entry to set the list of synchronously sampled analog input channels to acquire. This entry is a serialized C++ `std::vector<unsigned char>` having a size equal to the number of synchronously sampled analog input channels to acquire. Each entry of the vector contains the one-based channel number of the synchronously sampled analog input channel to acquire. The channels' data are delivered in the ordering they are specified here.

See the g.NEEDaccess Client API documentation for details on the effect of these values and how to configure them.

Client request for setting the configuration ([SET_CONFIGURATION](#))

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization:archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>15</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

**NOTE**

Each configuration belongs to a specific device. The payload is built of a serialized C++ `std::map<std::string, configuration>`. The `std::string` holds the device serial number.

The payload of the client's network API message in XML markup (The *g.USBamp* configuration list)

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <count>1</count> → [Indicates that only one device is stored in the std::vector]
    <item_version>0</item_version>
    <item class_id="1" tracking_level="0" version="0">
      <first>UA-2007.04.13</first> → [the device serial number]
      <second class_id="2" tracking_level="0" version="1"> → [the device specific configuration]
        <px class_id="4" class_name="gAPI::gUSBamp::Config::Config" tracking_level="1"
          version="0" object_id="_0">
          <BaseConfig class_id="5" tracking_level="0" version="0">
            <DeviceConfigurationBase class_id="3" tracking_level="1" version="0"
              object_id="_1"/>
          </BaseConfig>
          <AnalogOut class_id="6" tracking_level="0" version="0">
            <shape_>3</shape_>
            <frequency_>10</frequency_>
            <amplitude_>100</amplitude_>
            <offset_>0</offset_>
          </AnalogOut>
          <sample_rate_>256</sample_rate_>
          <number_of_scans_>0</number_of_scans_>
          <slave_>0</slave_>
          <short_cut_>0</short_cut_>
          <common_ground_ class_id="7" tracking_level="0" version="0">
            <TYPE_STD_VECTOR>
              <count>4</count>
              <item>0</item>
              <item>0</item>
              <item>0</item>
              <item>0</item>
            </TYPE_STD_VECTOR>
          </common_ground_>
          <common_reference_>
            <TYPE_STD_VECTOR>
              <count>4</count>
              <item>0</item>
              <item>0</item>
              <item>0</item>
              <item>0</item>
            </TYPE_STD_VECTOR>
          </common_reference_>
          <device_mode_>3</device_mode_>
          <scan_dio_>0</scan_dio_>
          <bandpass_>
            <count>16</count>
            <item_version>0</item_version>
            <item>-1</item>
            <item>-1</item>
            <item>-1</item>
          </bandpass_>
        </px>
      </second>
    </item>
  </value>
</boost_serialization>
```

```

    .
    .
    <item>-1</item>
    <item>-1</item>
    <item>-1</item>
  </bandpass_>
  <notch_>
    <count>16</count>
    <item_version>0</item_version>
    <item>-1</item>
    <item>-1</item>
    <item>-1</item>
    .
    .
    .
    <item>-1</item>
    <item>-1</item>
    <item>-1</item>
  </notch_>
  <bipolar_>
    <count>16</count>
    <item_version>0</item_version>
    <item>0</item>
    <item>0</item>
    <item>0</item>
    .
    .
    .
    <item>0</item>
    <item>0</item>
    <item>0</item>
  </bipolar_>
  <analog_in_ class_id="11" tracking_level="0" version="0">
    <TYPE_STD_VECTOR>
      <count>16</count> → [Indicates that 16 analog in channels are selected (in a 1:1 mapping)]
      <item_version>0</item_version>
      <item>1</item>
      <item>2</item>
      <item>3</item>
      .
      .
      .
      <item>14</item>
      <item>15</item>
      <item>16</item>
    </TYPE_STD_VECTOR>
  </analog_in_>
</px>
</second>
</item>
</value>
</boost_serialization>

```

Server reply

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>15</command_>
  <payload_></payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>

```

</boost_serialization>

3.3.3 CONTROL MESSAGES

The following table lists all additional control messages for g.USBamp devices only.

Value	Canonical name	Description
1	GET_BANDPASS_FILTERS	Get the list of available bandpass filters.
2	GET_NOTCH_FILTERS	Get the list of available notch filters.
3	GET_SCALING	Get scaling factor and offset values for each channel.
4	SET_SCALING	Set scaling factor and offset values for each channel.
5	GET_DIGITAL_IO	Get asynchronous digital I/O channel values.
6	SET_DIGITAL_OUTPUT	Set asynchronous digital output channels.
7	CALIBRATE	Calibrate a g.USBamp device.
8	GET_IMPEDANCE	This command is deprecated (use GET_IMPEDANCE_EX instead).
		Get impedance of a channel.
9	GET_DEVICE_INFO	Get information about the device.
10	GET_SUPPORTED_SAMPLING_RATES	Get the list of supported sampling rates.
12	GET_IMPEDANCE_EX	Get impedance of a channel.

Get the list of available bandpass filters (GET_BANDPASS_FILTERS)

Returns the list of available bandpass filters for the specified g.USBamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>1</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>1</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available filters is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>394</count> → [There are 394 filter entries in this list]
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <TypeId>1</TypeId>
    <Order>8</Order>
    <SamplingRate>32</SamplingRate>
    <LowerCutoffFrequency>0.100000000000000001</LowerCutoffFrequency>
    <UpperCutoffFrequency>0</UpperCutoffFrequency>
  </item>
  <item>
    <TypeId>1</TypeId>
    <Order>8</Order>
    <SamplingRate>32</SamplingRate> [A specific filter entry (at
index 1 in this case)]
    <LowerCutoffFrequency>1</LowerCutoffFrequency>
    <UpperCutoffFrequency>0</UpperCutoffFrequency>
  </item>
  <item>
    <TypeId>1</TypeId>
    <Order>8</Order>
    <SamplingRate>32</SamplingRate>
    <LowerCutoffFrequency>2</LowerCutoffFrequency>
    <UpperCutoffFrequency>0</UpperCutoffFrequency>
  </item>
  .
  .
  .
  <item>
    <TypeId>1</TypeId>
    <Order>8</Order>
    <SamplingRate>38400</SamplingRate>
    <LowerCutoffFrequency>5</LowerCutoffFrequency>
    <UpperCutoffFrequency>16000</UpperCutoffFrequency>
  </item>
</value>
</boost_serialization>
```



NOTE
The list of available filters is a serialized C++ `std::vector<...>`, where each entry represents a single filter structure. The filter structure has following layout:

Element	Corresponding C++ data type	Description												
TypeId	enum	The type of the filter. The following table lists all possible values:												
		<table><tr><th>Value</th><th>Canonical name</th><th>Description</th></tr><tr><td>0</td><td>FILTER_TYPE_CHEBYSHEV</td><td>A Chebyshev filter.</td></tr><tr><td>1</td><td>FILTER_TYPE_BUTTERWORTH</td><td>A Butterworth filter.</td></tr><tr><td>2</td><td>FILTER_TYPE_BESSEL</td><td>A Bessel filter.</td></tr></table>	Value	Canonical name	Description	0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.	1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.	2	FILTER_TYPE_BESSEL	A Bessel filter.
		Value	Canonical name	Description										
		0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.										
		1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.										
2	FILTER_TYPE_BESSEL	A Bessel filter.												
Order	double	The order of the filter.												
SamplingRate	double	The sampling rate that the filter is designed for in hertz.												
LowerCutoffFrequency	double	The lower cutoff frequency of the filter in hertz.												
UpperCutoffFrequency	Double	The upper cutoff frequency of the filter in hertz.												

If a filter should be used for a specific channel, the zero-based index of a particular filter entry within the list of available filters is the index of the filter to specify in the configuration structure for that channel.

Get the list of available notch filters (GET_NOTCH_FILTERS)

Returns the list of available notch filters for the specified g.USBamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>2</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>2</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available filters is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>20</count>  → [There are 20 filter entries in this list]
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <TypeId>1</TypeId>
    <Order>4</Order>
    <SamplingRate>128</SamplingRate>
    <LowerCutoffFrequency>48</LowerCutoffFrequency>
    <UpperCutoffFrequency>52</UpperCutoffFrequency>
  </item>
  <item>
    <TypeId>1</TypeId>
    <Order>4</Order>
    <SamplingRate>128</SamplingRate> } [A specific filter entry (at
index 1 in this case)]
    <LowerCutoffFrequency>58</LowerCutoffFrequency>
    <UpperCutoffFrequency>62</UpperCutoffFrequency>
  </item>
  <item>
    <TypeId>1</TypeId>
    <Order>4</Order>
    <SamplingRate>256</SamplingRate>
    <LowerCutoffFrequency>48</LowerCutoffFrequency>
    <UpperCutoffFrequency>52</UpperCutoffFrequency>
  </item>
  .
  .
  .
  <item>
    <TypeId>1</TypeId>
    <Order>4</Order>
    <SamplingRate>38400</SamplingRate>
    <LowerCutoffFrequency>58</LowerCutoffFrequency>
    <UpperCutoffFrequency>62</UpperCutoffFrequency>
  </item>
</value>
</boost_serialization>
```



NOTE

The list of available filters is a serialized C++ `std::vector<...>`, where each entry represents a single filter structure. The filter structure has following layout:

Element	Corresponding C++ data type	Description
TypeId	enum	The type of the filter. The following table lists all possible values:
Order	double	The order of the filter.
SamplingRate	double	The sampling rate that the filter is designed for in hertz.
LowerCutoffFrequency	double	The lower cutoff frequency of the filter in hertz.
UpperCutoffFrequency	double	The upper cutoff frequency of the filter in hertz.

If a filter should be used for a specific channel, the zero-based index of a particular filter entry within the list of available filters is the index of the filter to specify in the configuration structure for that channel.

Get scaling factor and offset values for each channel (GET_SCALING)

Retrieves the scaling values for the specified g.USBamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID. The request will fail if data acquisition is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>3</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>3</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The scaling factor and offset values are returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <scale_ class_id="1" tracking_level="0" version="0">
    <TYPE_STD_VECTOR>
      <count>16</count> → [The number of channels of a g.USBamp (always 16)]
      <item_version>0</item_version>
      <item>1</item>
      <item>1.0465479</item>
      <item>0.89800763</item>
      .
      .
      .
      <item>0.61243421</item>
      <item>0.61051887</item>
      <item>0.57678193</item>
    </TYPE_STD_VECTOR>
  </scale_>
  <offset_>
    <TYPE_STD_VECTOR>
      <count>16</count> → [The number of channels of a g.USBamp (always 16)]
      <item_version>0</item_version>
      <item>96.067947</item>
      <item>-335.94586</item>
      .
      .
      .
      <item>107.42146</item>
      <item>-223.65991</item>
      <item>174.54723</item>
    </TYPE_STD_VECTOR>
  </offset_>
</value>
</boost_serialization>

```

**NOTE**

The parameter entry is a serialized C++ structure containing two serialized C++ `std::vector<float>`. The first vector (`<scale_>`) holds the scaling factor values, and the second vector (`<offset_>`) holds the offset values for each channel in microvolts. A scaling factor or an offset value at index *i* within the vector corresponds to the *i*'th channel of the device. Scaling factor and offset values are always returned for all channels of the device.

Set scaling factor and offset values for each channel (SET_SCALING)

Sets the scaling values for the specified g.USBamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID. The request will fail if data acquisition is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>4</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the client's request holds a device-specific data entry with one parameter in its parameter list. The first item of the request's parameter vector of the device-specific data entry must be set to the scaling factor and offset values to set for the device (see below).

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <scale_ class_id="1" tracking_level="0" version="0">
    <TYPE_STD_VECTOR>
      <count>16</count> → [The number of channels of a g.USBamp (always 16)]
      <item_version>0</item_version>
      <item>1</item>
      <item>1.0465479</item>
      <item>0.89800763</item>
      .
      .
      .
      <item>0.61243421</item>
      <item>0.61051887</item>
      <item>0.57678193</item>
    </TYPE_STD_VECTOR>
  </scale_>
  <offset_>
    <TYPE_STD_VECTOR>
      <count>16</count> → [The number of channels of a g.USBamp (always 16)]
      <item_version>0</item_version>
      <item>96.067947</item>
      <item>-335.94586</item>
      <item>-650.38422</item>
      .
      .
      .
      <item>107.42146</item>
      <item>-223.65991</item>
      <item>174.54723</item>
    </TYPE_STD_VECTOR>
  </offset_>
</value>
</boost_serialization>

```

**NOTE**

The parameter entry is a serialized C++ structure containing two serialized C++ `std::vector<float>`. The first vector (`<scale_>`) holds the scaling factor values, and the second vector (`<offset_>`) holds the offset values for each channel in microvolts. A scaling factor or an offset value at index *i* within the vector corresponds to the *i*'th channel of the device. Scaling factor and offset values are always specified for all channels of the device.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with an empty parameter vector.

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>4</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

Get asynchronous digital I/O channel values (GET_DIGITAL_IO)

Retrieves the current values of all asynchronous digital input and output channels for the specified g.USBamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>5</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>4</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of all available asynchronous digital I/O channels and their current values is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>4</count> —————> [Is always 4 because there are 4 digital
i/o units]
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <ChannelNumber>1</ChannelNumber>
    <Direction>0</Direction>
    <Value>0</Value>
  </item>
  <item>
    <ChannelNumber>2</ChannelNumber>
    <Direction>0</Direction>
    <Value>0</Value>
  </item>
  <item>
    <ChannelNumber>1</ChannelNumber>
    <Direction>1</Direction>
    <Value>0</Value>
  </item>
  <item>
    <ChannelNumber>2</ChannelNumber>
    <Direction>1</Direction>
    <Value>0</Value>
  </item>
</value>
</boost_serialization>
```

(direction: input, value: false)]

[Represents a single I/O channel]



NOTE

The list of all available asynchronous digital I/O channels is a serialized C++ `std::vector<...>`, where each entry contains a single digital I/O channel structure. The digital I/O channel structure has the following layout:

Element	Canonical name	Description									
ChannelNumber	unsigned int	The channel number of the asynchronous digital I/O channel.									
Direction	enum	The direction of the corresponding digital channel. <table><tr><th>Value</th><th>Canonical name</th><th>Description</th></tr><tr><td>0</td><td>DIRECTION_IN</td><td>An input channel.</td></tr><tr><td>1</td><td>DIRECTION_OUT</td><td>An output channel.</td></tr></table>	Value	Canonical name	Description	0	DIRECTION_IN	An input channel.	1	DIRECTION_OUT	An output channel.
Value	Canonical name	Description									
0	DIRECTION_IN	An input channel.									
1	DIRECTION_OUT	An output channel.									
Value	bool	The current value of the corresponding channel. Possible values are 1 (HIGH) or 0 (LOW).									

Set asynchronous digital output channels (SET_DIGITAL_OUTPUT)

Sets or changes the values of the specified asynchronous digital output channels for the specified g.USBamp device in the data acquisition session associated with the specified session ID.

Only the output channels specified in the request are modified, while others preserve their current state.

g.USBamp version 2.0 devices accept channel numbers from 1 to 2.

g.USBamp version 3.0 devices accept channel numbers from 1 to 4.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if a channel entry to set contains invalid values or an invalid channel number.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>6</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the client's request holds a device-specific data entry with one parameter in its parameter list. The first item of the request's parameter vector of the device-specific data entry must be set to the list of digital output channels to change (see below).

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>2</count>
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <ChannelNumber>1</ChannelNumber>
    <Direction>1</Direction>
    <Value>1</Value>
  </item>
  <item>
    <ChannelNumber>2</ChannelNumber>
    <Direction>1</Direction>
    <Value>1</Value>
  </item>
</value>
</boost_serialization>
```

**NOTE**

The parameter entry is a serialized C++ `std::vector<...>` containing a single digital I/O channel structure entry for each asynchronous digital output channel whose value should be changed. The digital I/O channel structure has the following layout:

Element	Corresponding C++ data type	Description									
ChannelNumber	unsigned int	The channel number of the asynchronous digital output channel to modify.									
Direction	enum	The direction of the corresponding digital channel. Only output channels are allowed to be specified here. <table> <thead> <tr> <th>Value</th><th>Canonical name</th><th>Description</th></tr> </thead> <tbody> <tr> <td>0</td><td>DIRECTION_IN</td><td>An input channel.</td></tr> <tr> <td>1</td><td>DIRECTION_OUT</td><td>An output channel.</td></tr> </tbody> </table>	Value	Canonical name	Description	0	DIRECTION_IN	An input channel.	1	DIRECTION_OUT	An output channel.
Value	Canonical name	Description									
0	DIRECTION_IN	An input channel.									
1	DIRECTION_OUT	An output channel.									
Value	bool	The value to change the corresponding digital output channel to. Possible values are 1 (HIGH) or 0 (LOW).									

In this example, UA-2007.04.13 is a version 2.0 device. Both digital output channels (Direction = 1) are set to true (Value = 1).

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```



NOTE

The payload of the server's reply holds a device-specific data entry with an empty parameter vector.

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>6</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

Calibrate a g.USBamp device (CALIBRATE)

Calculates and returns new scaling values for each channel under the current conditions for the specified g.USBamp device in the data acquisition session associated with the specified session ID.

The calculated and returned scaling values are not applied on the device yet. Use [SET SCALING](#) with the returned scaling structure to apply them on the device.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID. The request will fail if data acquisition is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>7</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>7</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The calculated scaling and offset values are returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <scale_ class_id="1" tracking_level="0" version="0">
    <TYPE_STD_VECTOR>
      <count>16</count>
      <item_version>0</item_version>
      <item>1</item>
      <item>1.1300458</item>
      <item>1.1191366</item>
      .
      .
      .
      <item>0.51995999</item>
      <item>0.55989265</item>
      <item>0.55893165</item>
    </TYPE_STD_VECTOR>
  </scale_>
  <offset_>
    <TYPE_STD_VECTOR>
      <count>16</count>
      <item_version>0</item_version>
      <item>147.24617</item>
      <item>145.47035</item>
      <item>-307.95767</item>
      .
      .
      .
      <item>124.51715</item>
      <item>-221.49213</item>
      <item>176.81406</item>
    </TYPE_STD_VECTOR>
  </offset_>
</value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ structure containing two serialized C++ `std::vector<float>`. The first vector (`<scale_>`) holds the scaling values, and the second vector (`<offset_>`) holds the offset values for each channel in microvolts. A scaling or an offset value at index *i* within the vector corresponds to the *i*'th channel of the device. Scaling and offset values are always returned for all channels of the device.

Get impedance of a channel (GET_IMPEDANCE)

Measures and returns the impedance at the specified channel of the specified g.USBamp device in the data acquisition session associated with the specified session ID.

This command is deprecated (use [GET_IMPEDANCE_EX](#) instead).

All grounds are connected to the common ground during impedance measurement. Impedance is measured between the specified channel and ground. Impedance can be measured across ordinary synchronous analog input channels and each group's reference channel. The following table lists valid values for the channel number:

Channel value	Corresponding C++ data type
1 to 16	Synchronously sampled analog input channels 1 to 16.
17	Reference channel of group A (REFA).
18	Reference channel of group B (REFB).
19	Reference channel of group C (REFC).
20	Reference channel of group D (REFD).

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID. The request will fail if data acquisition is currently running. The request will fail if an invalid channel number is specified.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>8</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the client's request holds a device-specific data entry with one parameter in its parameter list. The first item of the request's parameter vector of the device-specific data entry must be set to the one-based channel number of the device for which to measure impedance.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>3</value>
</boost_serialization>
```

**NOTE**

The parameter entry is a serialized C++ unsigned char containing the channel number for which to measure the impedance value.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>8</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The impedance value of the specified channel in ohm is returned as serialized C++ double in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>5008.79999999999993</value>
</boost_serialization>
```

Get information about the device (GET_DEVICE_INFO)

Retrieves extended information about the specified g.USBamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>9</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>9</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The device information string is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>SNR=UA-2007.04.13
HWVER=V2.0
VER=2.0.2
DT=3.3
AT=5.0a
CP=3.0
DSP=3.0.2
XIL=2.0
FX2=3.0.1
</value>
</boost_serialization>
```

**NOTE**

The device information string is a serialized C++ `std::string`.

Get the list of supported sampling rates (GET_SUPPORTED_SAMPLING_RATES)

Retrieves a list of sampling rates and their features supported by the specified g.USBamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>10</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>10</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of supported sampling rates is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>12</count>
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <first>32</first> → [A supported sampling rate]
    <second>1</second> → [The recommended number of scans for the corresponding
sampling rate]
  </item>
```

```

<item>
  <first>64</first>
  <second>2</second>
</item>
<item>
  <first>128</first>
  <second>4</second>
</item>
<item>
  <first>256</first>
  <second>8</second>
</item>
<item>
  <first>512</first>
  <second>16</second>
</item>
<item>
  <first>600</first>
  <second>18</second>
</item>
<item>
  <first>1200</first>
  <second>36</second>
</item>
<item>
  <first>2400</first>
  <second>72</second>
</item>
<item>
  <first>4800</first>
  <second>144</second>
</item>
<item>
  <first>9600</first>
  <second>256</second>
</item>
<item>
  <first>19200</first>
  <second>256</second>
</item>
<item>
  <first>38400</first>
  <second>256</second>
</item>
</value>
</boost_serialization>

```

**NOTE**

The list of supported sampling rates is a serialized C++ `std::map<uint32_t, uint16_t>` where each entry represents a supported configuration. The first element (`<first>`) of the pair contains a supported sampling rate, while the second element (`<second>`) contains the recommended number of scans for that sampling rate.

Get impedance of a channel (GET_IMPEDANCE_EX)

Measures and returns the impedance at the specified channel of the specified g.USBamp device in the data acquisition session associated with the specified session ID.

All grounds are connected to the common ground during impedance measurement. Impedance is measured between the specified channel and ground. Impedance can be measured across ordinary synchronous analog input channels and each group's reference channel. The following table lists valid values for the channel number:

Channel value	Corresponding C++ data type
1 to 16	Synchronously sampled analog input channels 1 to 16.
17	Reference channel of group A (REFA).
18	Reference channel of group B (REFB).
19	Reference channel of group C (REFC).
20	Reference channel of group D (REFD).

All returned impedance values are rounded to the nearest integral ohm value. Negative values represent different error codes.

The following table lists the recommended classification of returned impedances and error codes for passive gel electrodes:

Impedance value in ohm	Recommended classification for passive gel electrodes
< 7,500	Good impedance.
7,500...20,000	Medium impedance.
> 20,000	Bad impedance.

Impedances for active electrodes connected via a g.GAMMAbox cannot be measured with g.USBamp devices.

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Returned value (error code)	Description
-1	The impedance could not be calculated but is estimated to be very low.
-2	The impedance could not be calculated but is estimated to be very low.
-3	The impedance could not be calculated but is estimated to be very high.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID. The request will fail if data acquisition is currently running. The request will fail if an invalid channel number is specified.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>12</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the client's request holds a device-specific data entry with one parameter in its parameter list. The first item of the request's parameter vector of the device-specific data entry must be set to the one-based channel number of the device for which to measure impedance.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>3</value>
</boost_serialization>
```

**NOTE**

The parameter entry is a serialized C++ unsigned char containing the channel number for which to measure the impedance value.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>UA-2007.04.13</target_>
  <command_>12</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The impedance value of the specified channel in ohm is returned as serialized C++ double in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>5009</value>
</boost_serialization>
```

3.4 G.HIAMP SPECIFICS

The messages described in this section only apply to g.HIamp devices.

3.4.1 INTRODUCTION TO THE G.HIAMP CONFIGURATION

g.HIamp devices can be configured with the commonly available [SET CONFIGURATION](#) control message. In turn, the currently set configuration can be read with the commonly available [GET CONFIGURATION](#) control message request.

The payload of the [SET CONFIGURATION](#) control messages, and the server's reply to a [GET CONFIGURATION](#) request, contain the escaped XML representation of the g.HIamp configuration (also referred to as *g.HIamp configuration list*), which is described in the following. The g.NEEDaccess Client API documentation and the g.HIamp instructions for use provide detailed descriptions about the particular configuration parameters and their effects, as well as valid values.



ATTENTION

The description of the payload content (i.e. the *g.HIamp configuration list*) of the message is given as non-escaped XML statement here. When sending a message, that XML statement must generally be converted to escaped XML syntax first before setting it as payload. When receiving a message, the escaped XML syntax in the payload field must be converted to a non-escaped XML statement.

The g.HIamp device configuration

The *g.HIamp configuration list* is the key element in this message. The configuration of a g.HIamp device is built upon various elements:

- Simple entries:

Element	Corresponding C++ data type	Description
SamplingRate	unsigned short	The sampling rate of the synchronously sampled inputs in hertz. Use GET SUPPORTED SAMPLING RATES for a list of valid values.

		Sampling Rate [Hz]	Recommended value for NumberOfScans
		256	8
		512	16
		600	32
		1200	64
		2400	128
		4800	256
		9600	256
NumberOfScans	unsigned	19200	256
	short	38400	256
		The number of scans to retrieve from the device at once, or zero to apply an automatically determined number of scans for the specified sampling rate. Setting <i>NumberOfScans</i> to a value other than zero is recommended for advanced users only. This value depends on the value of <i>SamplingRate</i>. The following table lists the recommended values of <i>NumberOfScans</i> for all possible sample rates.	
TriggerLinesEnabled	int	Indicates whether the synchronously sampled digital trigger input channels should be included in data acquisition. If enabled, the values of the digital trigger input channels come encoded in a single value as an additional channel appended to the analog channels. For g.HIamp, there are 16 trigger lines encoded in this value. The eight least significant bits (bits 1 to 8) contain the digital values (0 for LOW or 1 for HIGH) for trigger channels 1 to 8. Bits 17 to 24 contain the digital values of trigger channels 9 to 16. Bits 9 to 16 and bits 25 to 32 of the value are not used.	
HoldEnabled	int	Indicates whether the signal hold input is enabled. If the signal hold input is enabled (1), a HIGH level on the HOLD input causes the amplifier to latch the input channels' current values and ignore the actually applied signal until the HOLD input is pulled down to LOW again. If the signal hold input is disabled (0), the signal applied to the HOLD input has no effect.	
IsSlave	int	Indicates whether the device is operated in slave mode. If slave mode is disabled, the device is operated in master mode. Slave mode is not supported yet, so this value must always be 0.	

CounterEnabled	int	Indicates whether a sample counter should be applied on the very first channel selected for acquisition. If enabled (1), a sample counter is applied to the very first channel that is selected for acquisition. The actually applied signal on that channel is then disregarded.
----------------	-----	---

- The InternalSignalGenerator entry to configure the internal signal generator with the following elements:

Element	Corresponding C++ data type	Description						
Enabled	int	Indicates whether the internal signal generator should be enabled.						
WaveShape	enum	The wave shape of the signal generator. Valid values are: <table> <tr> <th>Value</th><th>Canonical name</th><th>Description</th></tr> <tr> <td>0</td><td>WAVESHAPE_SQUARE</td><td>A square wave.</td></tr> </table>	Value	Canonical name	Description	0	WAVESHAPE_SQUARE	A square wave.
Value	Canonical name	Description						
0	WAVESHAPE_SQUARE	A square wave.						
Amplitude	double	The amplitude of the signal generator in microvolts. This value must always be set to 7622.83.						
Frequency	unsigned short	The frequency of the signal generator in hertz. Valid values range from 1 Hz to 700 Hz.						
Offset	double	The offset of the signal generator in microvolts. This value must always be set to -7622.83.						

- The Channels entry to configure the analog input channels. The channels are represented as a serialized C++ array of size 256. Each array entry consist of the following elements:

Element	Corresponding C++ data type	Description
ChannelNumber	unsigned short	The one-based channel number that this configuration belongs to. The channel number must be ascending without gaps across all entries starting from 1.
Acquire	int	Indicates whether the channel that this configuration belongs to should be included in data acquisition.
BandpassFilterIndex	int	The zero-based index of the bandpass filter to use for the channel. The index is related to the list of available bandpass filters that are returned by GET_BANDPASS_FILTERS . A value of -1 indicates that no bandpass filter is used for the channel.
NotchFilterIndex	int	The zero-based index of the notch filter to use for the channel. The index is related to the list of available notch filters that are returned by GET_NOTCH_FILTERS . A value of -1 indicates that no notch filter is used for the channel.
ReferenceChannel	unsigned short	The one-based number of the channel to use as a reference for the current channel, or zero if no reference should be used for the current channel.

See the g.NEEDaccess Client API documentation for details on the effect of these values and how to configure them.

Client request for setting the configuration ([SET_CONFIGURATION](#))

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>15</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```



ATTENTION

Each configuration belongs to a specific device. The payload is built of a serialized C++ `std::map<std::string, configuration>`. The `std::string` holds the device serial number.

The payload of the client's network API message in XML markup (the *g.Hlamp* configuration list)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <count>1</count> → [Indicates that only one device is stored in the std::vector]
    <item_version>0</item_version>
    <item class_id="1" tracking_level="0" version="0">
      <first>HA-2012.04.16</first> → [The first element of the std::pair = the device serial number]
      <second class_id="2" tracking_level="0" version="1">
        <px class_id="4" class_name="gAPI::gHIamp::Configuration" tracking_level="1"
          version="0" object_id="_0">
          <DeviceConfigurationBase class_id="3" tracking_level="1" version="0"
            object_id="_1"></DeviceConfigurationBase>
          <deviceConfiguration_ class_id="5" tracking_level="0" version="0">
            <SamplingRate>256</SamplingRate>
            <NumberOfScans>0</NumberOfScans>
            <TriggerLinesEnabled>0</TriggerLinesEnabled>
            <HoldEnabled>0</HoldEnabled>
            <IsSlave>0</IsSlave>
            <CounterEnabled>1</CounterEnabled>
            <InternalSignalGenerator class_id="6" tracking_level="0" version="0">
              <Enabled>0</Enabled>
              <WaveShape>1</WaveShape>
              <Amplitude>7622.829999999999</Amplitude>
              <Frequency>10</Frequency>
              <Offset>-7622.829999999999</Offset>
            </InternalSignalGenerator>
          <Channels>
            <count>256</count> → [Must always specify 256 channels (the maximum a g.Hlamp may
provide)]
            <item class_id="7" tracking_level="0" version="0">
              <ChannelNumber>1</ChannelNumber>
              <Acquire>1</Acquire>
              <BandpassFilterIndex>-1</BandpassFilterIndex>
              <NotchFilterIndex>-1</NotchFilterIndex>
              <ReferenceChannel>3</ReferenceChannel>
            </item>
            <item>
              <ChannelNumber>2</ChannelNumber>
              <Acquire>1</Acquire>
              <BandpassFilterIndex>-1</BandpassFilterIndex>
              <NotchFilterIndex>-1</NotchFilterIndex>
              <ReferenceChannel>3</ReferenceChannel>
            </item>
            .
            .
            .
            <item>
              <ChannelNumber>255</ChannelNumber>
              <Acquire>0</Acquire>
              <BandpassFilterIndex>-1</BandpassFilterIndex>
              <NotchFilterIndex>-1</NotchFilterIndex>
              <ReferenceChannel>0</ReferenceChannel>
            </item>
          </Channels>
        </deviceConfiguration_>
      </second>
    </item>
  </value>
</boost_serialization>
```

```

        <item>
          <ChannelNumber>256</ChannelNumber>
          <Acquire>0</Acquire>
          <BandpassFilterIndex>-1</BandpassFilterIndex>
          <NotchFilterIndex>-1</NotchFilterIndex>
          <ReferenceChannel>0</ReferenceChannel>
        </item>
      </Channels>
    </deviceConfiguration_>
  </px>
</second>
</item>
</value>
</boost_serialization>

```

Server reply

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>15</command_>
  <payload_></payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>

```

3.4.2 CONTROL MESSAGES

The following table lists all additional control messages for g.Hlamp devices only.

Value	Canonical name	Description
1	GET_AVAILABLE_CHANNELS	Get available channels at the device.
2	GET_SUPPORTED_SAMPLING_RATES	Get the list of supported sampling rates.
3	GET_BANDPASS_FILTERS	Get the list of available bandpass filters.
4	GET_NOTCH_FILTERS	Get the list of available notch filters.
5	GET_FACTORY_SCALING	Get factory-set scaling factor and offset values for each channel.
6	GET_SCALING	Get scaling factor and offset values for each channel.
7	SET_SCALING	Set scaling factor and offset values for each channel.
8	CALIBRATE	Calibrate a g.Hlamp device.
9	GET_IMPEDANCE	This command is deprecated (use GET_IMPEDANCE_EX instead).
		Get impedance of all channels.
10	GET_DEVICE_INFO	Get information about the device.
11	GET_IMPEDANCE_EX	Get impedance of all channels.

A detailed description of these messages follows.

Get available channels at the device (GET_AVAILABLE_CHANNELS)

Gets a Boolean flag for each possible channel that a device in its full configuration may provide, indicating whether the corresponding channel is actually provided and available on the specified g.Hlamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>1</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>1</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available channels is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
    <count>256</count>      [The maximum number of channels of a fully
equipped g.HIamp (always 256)]
    <item_version>0</item_version>
    <item class_id="1" tracking_level="0" version="0">
        <first>1</first>
        <second>1</second>
    </item>
    <item>
        <first>2</first> → [The channel number]
        <second>1</second> → [The Boolean value (0 or 1) indicating whether the channel is
available or not]
    </item>
    .
    .
    .
    <item>
        <first>255</first>
        <second>1</second>
    </item>
    <item>
        <first>256</first>
        <second>1</second>
    </item>
</value>
</boost_serialization>

```

} *[A single channel's entry]*



NOTE

The list of available channels is a serialized C++ `std::map<uint16_t, bool>`, where each entry represents a single channel that may be provided by the device. The first element (`<first>`) of the pair contains the one-based channel number that the current entry belongs to, the second element (`<second>`) indicates whether the channel is actually provided and available on the device (1) or not (0).

Get the list of supported sampling rates (GET_SUPPORTED_SAMPLING_RATES)

Retrieves a list of sampling rates and their features supported by the specified g.Hlamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>2</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>2</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of supported sampling rates is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>9</count>
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <first>256</first> → [A supported sampling rate]
    <second>8</second> → [The recommended number of scans for the corresponding
sampling rate]
  </item>
  <item>
    <first>512</first>
    <second>16</second>
  </item>
  <item>
    <first>600</first>
    <second>18</second>
  </item>
  <item>
    <first>1200</first>
    <second>36</second>
  </item>
  <item>
    <first>2400</first>
    <second>72</second>
  </item>
  <item>
    <first>4800</first>
    <second>144</second>
  </item>
  <item>
    <first>9600</first>
    <second>256</second>
  </item>
  <item>
    <first>19200</first>
    <second>256</second>
  </item>
  <item>
    <first>38400</first>
    <second>256</second>
  </item>
</value>
</boost_serialization>
```



NOTE

The list of supported sampling rates is a serialized C++ `std::map<uint32_t, uint16_t>`, where each entry represents a supported configuration. The first element (`<first>`) of the pair contains a supported sampling rate, the second element (`<second>`) contains the recommended number of scans for that sampling rate.

Get the list of available bandpass filters (GET_BANDPASS_FILTERS)

Gets the list of available bandpass filters for the specified g.Hlamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>3</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>3</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available bandpass filters is returned in the first item of the reply's parameter vector of the device-specific data entry.

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available bandpass filters is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>170</count> There are 170 filter entries in this list
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <TypeId>1</TypeId>
    <Order>8</Order>
    <SamplingRate>256</SamplingRate>
    <LowerCutoffFrequency>0.100000000000000001</LowerCutoffFrequency>
    <UpperCutoffFrequency>0</UpperCutoffFrequency>
  </item>
  <item>
    <TypeId>1</TypeId>
    <Order>8</Order>
    <SamplingRate>256</SamplingRate>
    <LowerCutoffFrequency>1</LowerCutoffFrequency>
    <UpperCutoffFrequency>0</UpperCutoffFrequency>
  </item>
  .
  .
  .
  <item>
    <TypeId>1</TypeId>
    <Order>8</Order>
    <SamplingRate>4800</SamplingRate>
    <LowerCutoffFrequency>5</LowerCutoffFrequency>
    <UpperCutoffFrequency>1000</UpperCutoffFrequency>
  </item>
  <item>
    <TypeId>1</TypeId>
    <Order>8</Order>
    <SamplingRate>4800</SamplingRate>
    <LowerCutoffFrequency>5</LowerCutoffFrequency>
    <UpperCutoffFrequency>2000</UpperCutoffFrequency>
  </item>
</value>
</boost_serialization>

```

[A specific filter entry (at index 1 in this case)]



NOTE

The list of available filters is a serialized C++ `std::vector<...>`, where each entry represents a single filter structure. The filter structure has following layout:

Element	Corresponding C++ data type	Description												
TypeId	enum	The type of the filter. The following table lists all possible values:												
		<table><tr><th>Value</th><th>Canonical name</th><th>Description</th></tr><tr><td>0</td><td>FILTER_TYPE_CHEBYSHEV</td><td>A Chebyshev filter.</td></tr><tr><td>1</td><td>FILTER_TYPE_BUTTERWORTH</td><td>A Butterworth filter.</td></tr><tr><td>2</td><td>FILTER_TYPE_BESSEL</td><td>A Bessel filter.</td></tr></table>	Value	Canonical name	Description	0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.	1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.	2	FILTER_TYPE_BESSEL	A Bessel filter.
		Value	Canonical name	Description										
		0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.										
		1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.										
2	FILTER_TYPE_BESSEL	A Bessel filter.												
Order	double	The order of the filter												
SamplingRate	double	The sampling rate that the filter is designed for in hertz.												
LowerCutoffFrequency	double	The lower cutoff frequency of the filter in hertz.												
UpperCutoffFrequency	double	The upper cutoff frequency of the filter in hertz.												

If a filter should be used for a specific channel, the zero-based index of a particular filter entry within the list of available filters is the index of the filter to specify in the configuration structure for that channel.

Get the list of available notch filters (GET_NOTCH_FILTERS)

Gets the list of available notch filters for the specified g.Hlamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>4</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>4</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available notch filters is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>20</count> → [There are 20 filter entries in this list]
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <TypeId>1</TypeId>
    <Order>4</Order>
    <SamplingRate>128</SamplingRate>
    <LowerCutoffFrequency>48</LowerCutoffFrequency>
    <UpperCutoffFrequency>52</UpperCutoffFrequency>
  </item>
  <item>
    <TypeId>1</TypeId>
    <Order>4</Order>
    <SamplingRate>128</SamplingRate>
    <LowerCutoffFrequency>58</LowerCutoffFrequency>
    <UpperCutoffFrequency>62</UpperCutoffFrequency>
  </item>
  <item>
    <TypeId>1</TypeId>
    <Order>4</Order>
    <SamplingRate>256</SamplingRate>
    <LowerCutoffFrequency>48</LowerCutoffFrequency>
    <UpperCutoffFrequency>52</UpperCutoffFrequency>
  </item>
  .
  .
  .
  <item>
    <TypeId>1</TypeId>
    <Order>4</Order>
    <SamplingRate>38400</SamplingRate>
    <LowerCutoffFrequency>48</LowerCutoffFrequency>
    <UpperCutoffFrequency>52</UpperCutoffFrequency>
  </item>
  <item>
    <TypeId>1</TypeId>
    <Order>4</Order>
    <SamplingRate>38400</SamplingRate>
    <LowerCutoffFrequency>58</LowerCutoffFrequency>
    <UpperCutoffFrequency>62</UpperCutoffFrequency>
  </item>
</value>
</boost_serialization>

```

[A specific filter entry (at index 2 in this case)]



NOTE

The list of available filters is a serialized C++ `std::vector<...>`, where each entry represents a single filter structure. The filter structure has following layout:

Element	Corresponding C++ data type	Description												
TypeId	enum	The type of the filter. The following table lists all possible values.												
		<table><tr><th>Value</th><th>Canonical name</th><th>Description</th></tr><tr><td>0</td><td>FILTER_TYPE_CHEBYSHEV</td><td>A Chebyshev filter.</td></tr><tr><td>1</td><td>FILTER_TYPE_BUTTERWORTH</td><td>A Butterworth filter.</td></tr><tr><td>2</td><td>FILTER_TYPE_BESSEL</td><td>A Bessel filter.</td></tr></table>	Value	Canonical name	Description	0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.	1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.	2	FILTER_TYPE_BESSEL	A Bessel filter.
		Value	Canonical name	Description										
		0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.										
		1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.										
2	FILTER_TYPE_BESSEL	A Bessel filter.												
Order	double	The order of the filter.												
SamplingRate	double	The sampling rate that the filter is designed for in hertz.												
LowerCutoffFrequency	double	The lower cutoff frequency of the filter in hertz.												
UpperCutoffFrequency	double	The upper cutoff frequency of the filter in hertz.												

If a filter should be used for a specific channel, the zero-based index of a particular filter entry within the list of available filters is the index of the filter to specify in the configuration structure for that channel.

Get factory-set scaling factor and offset values for each channel (GET_FACTORY_SCALING)

Gets the scaling factor and offset values initially set by the manufacturer for the specified g.Hlamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID. The request will fail if data acquisition is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>5</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>5</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The factory-set scaling factor and offset values are returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <Factor>
    <count>256</count>
    <item>1.0028</item>
    <item>1.004011</item>
    .
    .
    .
    <item>1.0017231</item>
    <item>1.0036</item>
  </Factor>
  <Offset>
    <count>256</count>
    <item>-51.719269</item>
    <item>-159.22929</item>
    .
    .
    .
    <item>22.65823</item>
    <item>71.085693</item>
  </Offset>
</value>
</boost_serialization>

```

→ [The maximum number of channels of a fully equipped g.Hlamp (always 256)]

→ [The maximum number of channels of a fully equipped g.Hlamp (always 256)]

**NOTE**

The parameter entry is a serialized C++ structure containing two serialized `std::vector<float>`. The first vector (<Factor>) holds the factory-set scaling factor values, and the second vector (<Offset>) holds the factory-set offset values, for each channel in microvolts. A scaling factor or an offset value at index *i* within the vector corresponds to the *i*'th channel of the device. Scaling factor and offset values are always returned for all channels that a device may provide, and can be ignored for unavailable channels..

Get scaling factor and offset values for each channel (GET_SCALING)

Gets the currently set scaling factor and offset values of the specified g.Hlamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>6</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>6</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The currently set scaling factor and offset values are returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <Factor>
    <count>256</count>      → [The maximum number of channels of a fully
                             equipped g.Hlamp (always 256)]

    <item>1.0028</item>
    <item>1.004011</item>
    <item>1.004892</item>
    .
    .
    .
    <item>1.0017231</item>
    <item>1.0036</item>
  </Factor>
  <Offset>
    <count>256</count>      → [The maximum number of channels of a fully
                             equipped g.Hlamp (always 256)]

    <item>-51.719269</item>
    <item>-159.22929</item>
    .
    .
    .
    <item>22.65823</item>
    <item>71.085693</item>
  </Offset>
</value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ structure containing two serialized `std::vector<float>`. The first vector (`<Factor>`) holds the scaling factor values, and the second vector (`<Offset>`) holds the offset values, for each channel in microvolts. A scaling factor or an offset value at index *i* within the vector corresponds to the *i*'th channel of the device. Scaling factor and offset values are always returned for all channels that a device may provide, and can be ignored for unavailable channels.

Set scaling factor and offset values for each channel (SET_SCALING)

Sets the scaling factor and offset values for the specified g.Hlamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>7</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the client's request holds a device-specific data entry with one parameter in its parameter list. The first item of the request's parameter vector of the device-specific data entry must be set to the scaling factor and offset values to set for the device (see below).

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <Factor>
    <count>256</count>
    <item>1.0028</item>
    <item>1.004011</item>
    .
    .
    <item>1.0017231</item>
    <item>1.0036</item>
  </Factor>
  <Offset>
    <count>256</count>
    <item>-51.719269</item>
    <item>-159.22929</item>
    .
    .
    <item>22.65823</item>
    <item>71.085693</item>
  </Offset>
</value>
</boost_serialization>

```

→ [The maximum number of channels of a fully equipped g.Hlamp (always 256)]

→ [The maximum number of channels of a fully equipped g.Hlamp, always 256]

**NOTE**

The parameter entry is a serialized C++ structure containing two serialized `std::vector<float>`. The first vector (<Factor>) holds the scaling factor values, and the second vector (<Offset>) holds the offset values, for each channel in microvolts. A scaling factor or an offset value at index *i* within the vector corresponds to the *i*'th channel of the device. Scaling factor and offset values are always specified for all channels that a device may provide, and can be ignored for unavailable channels.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with an empty parameter vector.

The payload entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>7</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

Calibrate a g.Hlamp device (CALIBRATE)

Calculates, but does not yet apply, new scaling values for each synchronously sampled analog input channel under the current conditions, if the calibration set is applied for the specified g.Hlamp device in the data acquisition session associated with the specified session ID.

All channels must be shorted to ground (GND). This is done by applying the calibration equipment consisting of four connectors, one for each group (A, B, C, and D). They connect all channel inputs in a group with ground (GND).

The returned values will not be applied unless they're explicitly set with [SET_SCALING](#).

Calibration is required if amplitude and/or offset of a known input signal differs from the measured signal. The function returns *NaN* values (not a number) as the scaling factor and offset for channels that failed calibration.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID. The request will fail if data acquisition is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>8</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>8</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>2</count>
    <item_version>0</item_version>
    <item>
      → insert the FIRST parameter entry in escaped XML here
    </item>
    <item>
      → insert the SECOND parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

The payload of the server's reply holds a device-specific data entry with two parameters in its parameter list. The new scaling values calculated under the current conditions for each synchronously sampled analog input channel are returned in the first item of the reply's parameter vector of the device-specific data entry. Channels that failed calibration are returned in the second item of the reply's parameter vector of the device-specific data entry. If one or more channels fail calibration, the operation is still considered successful and the status field will indicate success.

The FIRST parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <Factor>
    <count>256</count>
    <item>0.99676788</item>
    <item>0.99397445</item>
    .
    .
    .
    <item>nan</item>      → [Channel 254 failed calibration and contains
                           nan instead]
    <item>nan</item>      → [Channel 255 failed calibration and contains
                           nan instead]
    <item>0.99908137</item>
  </Factor>
  <Offset>
    <count>256</count>
    <item>131.15103</item>
    <item>188.37189</item>
    .
    .
    .
    <item>nan</item>      → [Channel 254 failed calibration and contains
                           nan instead]
    <item>nan</item>      → [Channel 255 failed calibration and contains
                           nan instead]
    <item>172.77513</item>
  </Offset>
</value>
</boost_serialization>
```



NOTE

The first parameter entry is a serialized C++ structure containing two serialized C++ `std::vector<float>`. The first vector (`<Factor>`) holds the new scaling factor values, and the second vector (`<Offset>`) holds the new offset values, calculated under the current conditions for each synchronously sampled analog input channel in microvolts. A scaling factor or an offset value at index i within the vector corresponds to the i 'th channel of the device. Scaling factor and offset values are always returned for all channels that a device may provide, and can be ignored for unavailable channels.

If a channel fails calibration, its corresponding scaling factor and offset value will hold `nan` and the number of that channel is included in the list of channels that failed calibration (see the second parameter below).

In the given example, channels 254 and 255 failed calibration.

The SECOND parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>
  <count>2</count>
  <item_version>0</item_version>
  <item>254</item>
  <item>255</item>
</value>
</boost_serialization>
```

**NOTE**

The first parameter entry is a serialized C++ `std::vector<uint16_t>`, where each item contains the one-based number of a channel that failed calibration. In the given example, channels 254 and 255 failed calibration.

Get impedance of all channels (`GET_IMPEDANCE`)

Measures impedance of all available synchronously sampled analog input channels within the specified g.Hlamp device in the data acquisition session associated with the specified session ID for active or passive electrodes in ohm.

This command is deprecated (use [GET_IMPEDANCE_EX](#) instead).

Channel 1 and the ground electrode must always be connected for impedance measurement. A special z-electrode must be used on channel 1 if active electrodes are used. Active and passive electrodes cannot be mixed.

Channel 1 plays a special role in impedance measurement and must always be connected. However, its impedance cannot be determined. Channel 1 will never be included in the returned list.

All returned impedance values are rounded to the nearest integral kilo-ohm value in range. All measured values are clipped within a range of 5,000 ohm to 500,000 ohm. Values below 5,000 ohm or above 500,000 ohm represent different error codes and do not rate the severeness of error. Thus, they are not comparable to each other. The returned list only includes available channels that yielded a finite impedance value, except channel 1.

If a TMS box is connected, the caller must subtract 10 kilo-ohm from valid impedance values to get the correct value (do not do this for error codes).

The following table lists the recommended classification of returned impedances for passive gel electrodes:

Impedance value in ohm	Recommended classification for passive gel electrodes
< 5,000	Not measurable, estimated good impedance.
5,000...500,000	Measured impedance values.
> 500,000	Not measurable impedance due to an erroneous signal (maybe due to a not connected electrode, a too big offset, etc.).

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Returned value (error code)	Description
3,000	The impedance could not be calculated but is estimated to be very low.
4,000	The amplitude of the measured signal almost equals the original signal. The impedance is estimated to be very low.
501,000	The amplitude of the measured signal is too small. The impedance is estimated to be very high.
502,000	The offset of the measured signal is too large.
503,000	The RMS value of the measured signal is too large.
504,000	The low frequency power of the measured signal is too large.
505,000	The amplitude of the measured signal at channel 1 is too small.
506,000	The offset, RMS, or low frequency power at channel 1 is too large.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition is currently running.

The request will fail if the first parameter's list of active electrodes is not empty but does not contain all available channel numbers.

The request will not fail if an invalid reference signal is measured on channel 1 (maybe due to a not connected electrode). Instead, appropriate error codes will be assigned to the other channels.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>9</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

The payload of the client's request holds a device-specific data entry with one parameter in its parameter list. The first item of the request's parameter vector of the device-specific data entry must contain a list of one-based channel numbers that have an active electrode connected. All other channels are assumed to have passive electrodes connected.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>
  <count>64</count>
  <item_version>0</item_version>
  <item>1</item>
  <item>2</item>
  .
  .
  .
  <item>63</item>
  <item>64</item>
</value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ `std::vector<uint16_t>` containing only the one-based channel numbers that have active electrodes connected. In this example, channels 1 to 64 have active electrodes connected, and channels 65 to 256 (for a 256 channel device) have passive (or no) electrodes connected.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>9</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of measured impedances, for all available synchronously sampled analog input channels, is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
    <count>255</count>→ [The number of measured channels that the list
contains, excluding channel 1]
    <item_version>0</item_version>
    <item class_id="1" tracking_level="0" version="0">
        <first>2</first>
        <second>501000</second>
    </item>
    <item>
        <first>3</first>
        <second>501000</second>
    </item>
    .
    .
    .
    <item>
        <first>255</first>
        <second>501000</second>
    </item>
    <item>
        <first>256</first>
        <second>501000</second>
    </item>
</value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ `std::map<uint16_t, double>`, where each `<item>` entry contains the one-based channel number in its key (`<first>`) and the measured impedance value of that channel in ohm in its value (`<second>`). Only available channels are measured and returned, excluding channel 1.

Get information about the device (GET_DEVICE_INFO)

Retrieves extended information about the specified g.Hlamp device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>10</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>10</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The device information string is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>SERIAL:HA-2012.04.16
VERSION:1.0
MODULES:111111111111111111
VER=1.0.6
A1.1=1.4a
A1.2=1.4a
A1.3=1.4a
A1.4=1.4a
A1.5=1.4a
A1.6=1.4a
A1.7=1.4a
A1.8=1.4a
A1.9=1.4a
A1.10=1.4a
A1.11=1.4a
A1.12=1.4a
A1.13=1.4a
A1.14=1.4a
A1.15=1.4a
A1.16=1.4a
A2=0.3a
A3=0.2
A10=0.2
FW.DSP=1.2
FW.ARM=1.2
FW.FPGA=3.0
</value>
</boost_serialization>

```

**NOTE**

The device information string is a serialized C++ `std::string`.

Get impedance of all channels (GET_IMPEDANCE_EX)

Measures impedance of all available synchronously sampled analog input channels within the specified g.Hlamp device in the data acquisition session associated with the specified session ID for active or passive electrodes in ohm.

Channel 1 and the ground electrode must always be connected for impedance measurement. A special z-electrode must be used on channel 1 if active electrodes are used. Active and passive electrodes cannot be mixed.

Channel 1 plays a special role in impedance measurement and must always be connected. However, its impedance cannot be determined. Channel 1 will never be included in the returned list.

If a TMS electrode connector box is connected, set the third parameter entry in the client's request payload to 1 accordingly; 10 kilo-ohm will be subtracted of all measured impedance values in this case due to internal dropping resistors. Hence, no other type of electrode connector boxes can be connected to the same device if an TMS electrode connector box is connected for impedance measurement.

All returned impedance values are rounded to the nearest integral ohm value in range. All measured values are clipped within a range of 600 ohm to 500,000 ohm. Negative values represent different error codes. The returned map only includes available channels that yielded a finite impedance value, except channel 1.

The following table lists the recommended classification of returned impedances for passive gel electrodes:

Impedance value in ohm	Recommended classification for passive gel electrodes
< 7,500	Good impedance.
7,500...20,000	Medium impedance.
> 20,000	Bad impedance.

The following table lists the recommended classification of returned impedances for active gel electrodes:

Impedance value in ohm	Recommended classification for active gel electrodes
< 65,000	Good impedance.
65,000...200,000	Medium impedance.
> 200,000	Bad impedance.

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Returned value (error code)	Description
-1	The amplitude of the measured signal almost equals the original signal. The impedance is estimated to be very low.
-2	The impedance could not be calculated but is estimated to be very low.
-3	The amplitude of the measured signal is too small. The impedance is estimated to be very high.
-4	The offset of the measured signal is too big.
-5	The RMS value of the measured signal is too big.
-6	The drift of the measured signal is too big.
-9	The amplitude of the measured signal at channel 1 is too small.
-10	The offset, RMS, or drift at channel 1 is too big.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition is currently running.

The request will not fail if an invalid reference signal is measured on channel 1 (maybe due to a not connected electrode). Instead, appropriate error codes will be assigned to the other channels.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>11</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>3</count>
    <item_version>0</item_version>
    <item>
      → insert the FIRST parameter entry in escaped XML here
    </item>
    <item>
      → insert the SECOND parameter entry in escaped XML here
    </item>
    <item>
      → insert the THIRD parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

The payload of the client's request holds a device-specific data entry with three parameters in its parameter list.

The first item of the request's parameter vector of the device-specific data entry is an enumeration value, where a value of 0 represents all passive electrodes and a value of 1 represents all active electrodes.

The second item is a flag indicating whether a TMS electrode connector box is connected and impedance values of all channels should be corrected by 10 kilo-ohm due to its internal dropping resistors.

The third item is a flag indicating whether this is the start of a new measurement (i.e. the first in a series of consecutive calls to [GET IMPEDANCE EX](#)) and internal interim values of previous measurements should be cleared.

The FIRST parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>0</value>
</boost_serialization>
```

**NOTE**

The parameter entry is an integral number (a serialized C++ enumeration value) representing the type of the connected electrodes. In this example, passive electrodes were selected. Valid values are:

Value	Canonical name	Description
0	ELECTRODE_TYPE_PASSIVE	Passive electrodes.
1	ELECTRODE_TYPE_ACTIVE	Active electrodes.

The SECOND parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>0</value>
</boost_serialization>
```

**NOTE**

The parameter entry is a serialized C++ `bool` value indicating whether a TMS electrode connector box is connected. Set a value of 1 if a TMS electrode connector box is connected and impedance values of all channels should be corrected by 10 kilo-ohm. Otherwise, set a value of 0.

The THIRD parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value>1</value>
</boost_serialization>
```

**NOTE**

The parameter entry is a serialized C++ `bool` value indicating the start of a new impedance measurement session after a pause. If this is start of a new impedance measurement session (i.e. the first in a series of consecutive [GET_IMPEDANCE_EX](#) requests), set this value to 1 to clear internal interim values of previous measurements. For the following repeated calls, set this value to 0 until the end of the measurement session.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>HA-2012.04.16</target_>
  <command_>11</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of measured impedances, for all available synchronously sampled analog input channels, is returned in the first item of the reply's parameter vector of the device-specific data entry.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>255</count>
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <first>2</first>
    <second>-3</second>
  </item>
  <item>
    <first>3</first>
    <second>-3</second>
  </item>
  .
  .
  .
  <item>
    <first>255</first>
    <second>-3</second>
  </item>
  <item>
    <first>256</first>
    <second>-3</second>
  </item>
</value>
</boost_serialization>
```

→ [The number of measured channels that the list contains, excluding channel 1]



NOTE

The parameter entry is a serialized C++ `std::map<uint16_t, double>`, where each `<item>` entry contains the one-based channel number in its key (`<first>`) and the measured impedance value (or error code) of that channel in ohm in its value (`<second>`). Only available channels are measured and returned, excluding channel 1.

3.5 G.NAUTILUS SPECIFICS

The messages described in this section only apply to g.Nautilus devices.

3.5.1 INTRODUCTION TO THE G.NAUTILUS CONFIGURATION

g.Nautilus devices can be configured with the commonly available [SET CONFIGURATION](#) control message. In turn, the currently set configuration can be read with the commonly available [GET CONFIGURATION](#) control message request.

The payload of the [SET CONFIGURATION](#) control messages, and the server's reply to a [GET CONFIGURATION](#) request, contain the escaped XML representation of the g.Nautilus configuration (also referred to as *g.Nautilus configuration list*), which is described below. The g.NEEDaccess Client API documentation and the g.Nautilus instructions for use provide detailed descriptions about the particular configuration parameters and their effects, as well as valid values.



ATTENTION

The description of the payload content (i.e. the g.Nautilus configuration list) of the message is given as non-escaped XML statement here. When sending a message, that XML statement must generally be converted to escaped XML syntax first before setting it as payload. When receiving a message, the escaped XML syntax in the payload field must be converted to non-escaped XML statement.

The g.Nautilus device configuration

The *g.Nautilus configuration list* is the key element in this message. The configuration of a g.Nautilus device is built upon various elements:

- Simple entries:

Element	Corresponding C++ data type	Description												
slave_	bool	Indicates whether the g.Nautilus device is operated in slave mode (1) or not (0). Slave mode is not supported yet, so this value must always be 0.												
input_signal_	enum	Specifies the input signal of all channels for the device. Valid values are: <table><tr><th>Value</th><th>Canonical name</th><th>Description</th></tr><tr><td>0</td><td>INPUT_SIGNAL_ELECTRODE</td><td>Analog input from electrodes.</td></tr><tr><td>1</td><td>INPUT_SIGNAL_SHORTCUT</td><td>Channels inputs shorted to GND.</td></tr><tr><td>5</td><td>INPUT_SIGNAL_TEST_SIGNAL</td><td>Test Signal as input for all electrodes.</td></tr></table>	Value	Canonical name	Description	0	INPUT_SIGNAL_ELECTRODE	Analog input from electrodes.	1	INPUT_SIGNAL_SHORTCUT	Channels inputs shorted to GND.	5	INPUT_SIGNAL_TEST_SIGNAL	Test Signal as input for all electrodes.
Value	Canonical name	Description												
0	INPUT_SIGNAL_ELECTRODE	Analog input from electrodes.												
1	INPUT_SIGNAL_SHORTCUT	Channels inputs shorted to GND.												
5	INPUT_SIGNAL_TEST_SIGNAL	Test Signal as input for all electrodes.												
number_of_scans_	unsigned int	The number of scans to retrieve from the device at once, or zero to apply an automatically determined number of scans for the specified sampling rate. Setting number_of_scans_ to a value other than zero is recommended for advanced users only. This value depends on the value of sample_rate_. The following table lists the recommended values of number_of_scans_ for all possible sampling rates. <table><tr><th>Sampling Rate [Hz]</th><th>Recommended value for NumberOfScans</th></tr><tr><td>256</td><td>8</td></tr><tr><td>512</td><td>15</td></tr></table>	Sampling Rate [Hz]	Recommended value for NumberOfScans	256	8	512	15						
Sampling Rate [Hz]	Recommended value for NumberOfScans													
256	8													
512	15													
sample_rate_	unsigned int	The sampling rate of the g.Nautilus device in hertz. Use GET_SUPPORTED_SAMPLING_RATES for a list of valid values.												
noise_reduction_	bool	Indicates whether noise reduction should be enabled.												
car_	bool	Indicates whether common average reference (CAR) calculation should be enabled.												
acceleration_data_	bool	Indicates whether acceleration data channels should be included in data acquisition. If enabled, the acceleration values of the x, y, and z axis come as three additional												

		channel appended to the analog channels. Acceleration values range within ± 6 g.
counter_	bool	Indicates whether the counter channel should be included in data acquisition. If enabled, counter comes as an additional channel appended to the acceleration data channels. The counter value is the number of the current sample.
link_quality_information	bool	Indicates whether the link quality information channel should be included in data acquisition. If enabled, link quality information comes as an additional channel appended to the counter channel. Link quality information values range within 0 and 100 percent.
battery_level_	bool	Indicates whether the battery level (state of charge) channel should be included in data acquisition. If enabled, battery level comes as an additional channel appended to the link quality information channel. Battery level values range within 0 and 100 percent.
digital_ios_	bool	Indicates whether the synchronously sampled digital input channels should be included in data acquisition. If enabled, the values of the digital trigger input channels come bit-encoded in a single additional channel appended to the battery level channel.
validation_indicator_	bool	Indicates whether the validation indicator channel should be included in data acquisition. If enabled, the value of the validation indicator comes as an additional channel appended to the digital IO channel. Valid validation indicator values are 0 and 1.
network_channel_	unsigned int	The radio channel used for wireless communication between headset and base station of the device.

- The `channel_configurations_` entry to configure the synchronously sampled analog input channels. The channels are represented as a serialized C++ `std::vector<...>` of size 64. Each vector entry consist of the following elements:

Element	Corresponding C++ data type	Description
channel_number_	unsigned int	The zero-based channel number that this configuration belongs to. The channel number must be ascending without gaps over all entries starting from 0.
enabled_	bool	Indicates whether the channel is enabled for data acquisition or not.
sensitivity_	double	The input sensitivity of the channel in microvolts.
used_for_noise_reduction_	bool	Indicates whether the channel is used for noise reduction computation.
used_for_car_	bool	Indicates whether the channel is used for common average reference (CAR) calculation.
bandpass_filter_index_	int	The zero-based index of the bandpass filter to use for the channel. The index is related to the list of available bandpass filters that are returned by GET_BANDPASS_FILTERS . A value of -1 indicates that no bandpass filter is used for the channel.
notch_filter_index_	int	The zero-based index of the notch filter to use for the channel. The index is related to the list of available notch filters that are returned by GET_NOTCH_FILTERS . A value of -1 indicates that no notch filter is used for the channel.
bipolar_channel_	int	The bipolar derivation setting for the channel. Set this value to -1 if unipolar derivation should be used with ground and reference electrodes. Otherwise, if bipolar derivation should be performed for the channel that this configuration belongs to, set \ref BipolarChannel to the zero-based channel number whose signal should be subtracted from the measured signal of this channel.

Client request for setting the configuration ([SET_CONFIGURATION](#))

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>15</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>

```

```
</boost_serialization>
```

See the g.NEEDaccess Client API documentation for details on the effect of these values and how to configure them.



NOTE

Each configuration belongs to a specific device. The payload is built of a serialized C++ `std::map<std::string, configuration>`. The `std::string` holds the device serial number.

The payload of the client's network API message in XML markup (the *g.Nautilus configuration list*)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <count>1</count> → [Indicates that only one device is stored in the std::vector]
    <item_version>0</item_version>
    <item class_id="1" tracking_level="0" version="0">
      <first>NB-2013.08.01</first> → [The first element of the std::pair = the device serial number]
      <second class_id="2" tracking_level="0" version="1">
        <px class_id="4" class_name="gAPI::gNautilus::Configuration"
          tracking_level="1" version="0" object_id="_0">
          <DeviceConfigurationBase class_id="3" tracking_level="1" version="0"
            object_id="_1"></DeviceConfigurationBase>
          <slave_>0</slave_>
          <input_signal_>0</input_signal_>
          <number_of_scans_>0</number_of_scans_>
          <sample_rate_>250</sample_rate_>
          <noise_reduction_>0</noise_reduction_>
          <car_>0</car_>
          <acceleration_data_>0</acceleration_data_>
          <counter_>1</counter_>
          <link_quality_information_>0</link_quality_information_>
          <battery_level_>0</battery_level_>
          <digital_ios_>0</digital_ios_>
          <validation_indicator_>0</validation_indicator_>
          <network_channel_>0</network_channel_>
          <channel_configurations_ class_id="5" tracking_level="0" version="0">
            <count>64</count>
            <item_version>0</item_version>
            <item class_id="6" tracking_level="0" version="0">
              <channel_number_>0</channel_number_>
              <enabled_>1</enabled_>
              <sensitivity_>187500</sensitivity_>
              <used_for_noise_reduction_>0</used_for_noise_reduction_>
              <used_for_car_>1</used_for_car_>
              <bandpass_filter_index_>-1</bandpass_filter_index_>
              <notch_filter_index_>-1</notch_filter_index_>
              <bipolar_channel_>-1</bipolar_channel_>
            </item>
            <item>
              <channel_number_>1</channel_number_>
              <enabled_>1</enabled_>
              <sensitivity_>187500</sensitivity_>
              <used_for_noise_reduction_>0</used_for_noise_reduction_>
              <used_for_car_>1</used_for_car_>
              <bandpass_filter_index_>-1</bandpass_filter_index_>
              <notch_filter_index_>-1</notch_filter_index_>
              <bipolar_channel_>-1</bipolar_channel_>
            </item>
          </channel_configurations_>
        </second>
      </item>
    </item>
  </value>
```

```

    </item>
    .
    .
    .
    <item>
      <channel_number_>62</channel_number_>
      <enabled_>0</enabled_>
      <sensitivity_>187500</sensitivity_>
      <used_for_noise_reduction_>0</used_for_noise_reduction_>
      <used_for_car_>1</used_for_car_>
      <bandpass_filter_index_>-1</bandpass_filter_index_>
      <notch_filter_index_>-1</notch_filter_index_>
      <bipolar_channel_>-1</bipolar_channel_>
    </item>
    <item>
      <channel_number_>63</channel_number_>
      <enabled_>0</enabled_>
      <sensitivity_>187500</sensitivity_>
      <used_for_noise_reduction_>0</used_for_noise_reduction_>
      <used_for_car_>1</used_for_car_>
      <bandpass_filter_index_>-1</bandpass_filter_index_>
      <notch_filter_index_>-1</notch_filter_index_>
      <bipolar_channel_>-1</bipolar_channel_>
    </item>
  </channel_configurations_>
</px>
</second>
</item>
</value>
</boost_serialization>

```

Server reply

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>15</command_>
  <payload_></payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>

```

3.5.2 CONTROL MESSAGES

The following table lists all additional control messages for g.Nautilus devices only.

Value	Canonical name	Description
1	GET_CHANNEL_NAMES	Get names of available channels.
2	SET_NETWORK	Set the radio channel to use for wireless communication between headset and base station.

3	GET_NETWORK	Get the currently used radio channel for wireless communication between headset and base station.
4	SET_SCALING	Set scaling factor and offset values for each channel.
5	GET_SCALING	Get scaling factor and offset values for each channel.
6	CALIBRATE	Calibrate a g.Nautilus device.
7	GET_BANDPASS_FILTERS	Get the list of available bandpass filters.
8	GET_NOTCH_FILTERS	Get the list of available notch filters.
9	RESET_SCALING	Reset scaling and offset values for each channel.
10	GET_IMPEDANCE	This command is deprecated (use GET_IMPEDANCE_EX instead). Get the impedance of all channels.
11	START_IMPEDANCE_MEASUREMENT	This command is deprecated (use START_IMPEDANCE_MEASUREMENT_EX instead). Start asynchronous impedance measurement.
12	STOP_IMPEDANCE_MEASUREMENT	Stop asynchronous impedance measurement.
14	GET_AVAILABLE_CHANNELS	Get available channels at the device.
15	GET_SUPPORTED_SAMPLING_RATES	Get the list of supported sampling rates.
16	GET_DIGITAL_IOS	Get available digital I/O channels at the device.
17	GET_SUPPORTED_NW_CHANNELS	Get the list of supported radio channels.
18	GET_SUPPORTED_SENSITIVITIES	Get the list of supported sensitivities.
19	GET_SUPPORTED_INPUT_SOURCES	Get the list of supported input signals.
20	GET_DEVICE_INFO	Get information about the device.
21	GET_IMPEDANCE_EX	Get the impedance of all channels.
22	START_IMPEDANCE_MEASUREMENT_EX	Start asynchronous impedance measurement.

A detailed description of these messages follows.

Get names of available channels (GET_CHANNEL_NAMES)

Retrieves the number of actually mounted modules and the names of all available channels at the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>1</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>1</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>2</count>
    <item_version>0</item_version>
    <item>
      → insert the FIRST parameter entry in escaped XML here
    </item>
    <item>
      → insert the SECOND parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with two parameters in its parameter list. The number of actually mounted modules is returned in the first item of the reply's parameter vector. The available channels and their names are returned in the second item of the reply's parameter vector.

The FIRST parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>4</value>
</boost_serialization>
```

**NOTE**

The first parameter entry is a serialized C++ uint₈ value containing the number of actually mounted modules in the device.

The SECOND parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>32</count>      → [The number of available channels]
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <first>0</first>      → [The zero-based channel number of an available channel]
    <second>FP1</second>  → [The electrode's name of the corresponding channel]
  </item>
  <item>
    <first>1</first>
    <second>FP2</second>
  </item>
  .
  .
  .
  <item>
    <first>30</first>
    <second>P08</second>
  </item>
  <item>
    <first>31</first>
    <second>OZ</second>
  </item>
</value>
</boost_serialization>
```

**NOTE**

The second parameter entry is a serialized C++ std::map<uint₃₂_t, std::string>, where each <item> entry contains the zero-based number of an available channel in its key (<first>) and the corresponding name in its value (<second>).

Set the radio channel to use for wireless communication between headset and base station (SET_NETWORK)

Sets the radio channel to use for wireless communication between the headset and base station for the specified g.Nautilus device in the data acquisition session associated with the specified session ID. Use [GET_SUPPORTED_NW_CHANNELS](#) to retrieve a list of valid radio channels.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID. The request will fail if an invalid radio channel is specified.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>2</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

The payload of the client's request holds a device-specific data entry with one parameter in its parameter list. The first item of the request's parameter vector of the device-specific data entry must be set to the radio channel number to use.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>22</value> → [The number of the selected radio channel]
</boost_serialization>
```



NOTE

The first parameter entry is a serialized C++ `uint8_t` containing the radio channel number to use.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>2</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

The payload of the server's reply holds a device-specific data entry with an empty parameter vector.

Get the currently used radio channel for wireless communication between headset and base station (GET_NETWORK)

Gets the current radio channel used for wireless communication between the headset and base station for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>3</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>3</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The number of the currently used radio channel is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>22</value> → [The currently used radio channel number]
</boost_serialization>
```

**NOTE**

The parameter entry is a serialized C++ `uint32_t` value containing the currently used radio channel number.

Set scaling factor and offset values for each channel (SET_SCALING)

Sets the scaling factor and offset values for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition or impedance measurement is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>4</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

The payload of the client's request holds a device-specific data entry with one parameter in its parameter list. The first item of the request's parameter vector of the device-specific data entry must be set to the scaling factor and offset values to set for the device (see below).

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <offset_>
    <count>64</count>
    <item>0</item>
    <item>0</item>
    <item>0</item>
    .
    .
    .
    <item>0</item>
    <item>0</item>
    <item>0</item>
  </offset_>
  <scaling_>
    <count>64</count>
    <item>1</item>
    <item>1</item>
    <item>1</item>
    .
    .
    .
    <item>1</item>
    <item>1</item>
    <item>1</item>
  </scaling_>
</value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ structure containing two serialized C++ float arrays of size 64. For each channel, the first array (<offset_>) holds the offset values in microvolts, and the second array (<scaling_>) holds the scaling factor values. A scaling factor or an offset value at index *i* within the array corresponds to the *i*'th channel of the device. Scaling factor and offset values are always specified for all channels that a device may provide in its full configuration, and can be ignored for unavailable channels.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>4</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with an empty parameter vector.

Get scaling factor and offset values for each channel (GET_SCALING)

Gets the currently set scaling factor and offset values of the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition or any impedance measurements are currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>5</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>5</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The currently set scaling factor and offset values are returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <offset_>
    <count>64</count>           → [The number of all channels, always 64]
    <item>24.91011</item>       → [The offset value for the first channel]
    <item>25.89259</item>       → [The offset value for the second channel]
    <item>24.945881</item>
    .
    .
    .
    <item>0</item>
    <item>0</item>
    <item>0</item>
  </offset_>
  <scaling_>
    <count>64</count>           → [The number of all channels, always 64]
    <item>1.001188</item>       → [The scale factor for the first channel]
    <item>1.001254</item>       → [The scale factor for the first channel]
    <item>1.001191</item>
    .
    .
    .
    <item>1</item>
    <item>1</item>
    <item>1</item>
  </scaling_>
</value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ structure containing two serialized C++ float arrays of size 64. For each channel, the first array (<offset_>) holds the offset values in microvolts, and the second array (<scaling_>) holds the scaling factor values. A scaling factor or an offset value at index i within the array corresponds to the i 'th channel of the device. Scaling factor and offset values are always specified for all channels that a device may provide in its full configuration, and can be ignored for unavailable channels.

Calibrate a g.Nautilus device (CALIBRATE)

Calculates, but does not yet apply, new scaling values for each synchronously sampled analog input channel under the current conditions, if the calibration set is applied for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The returned values will not be applied unless they're explicitly set with [SET_SCALING](#).

Calibration is required if amplitude and/or offset of a known input signal differ from the measured signal. The function returns *NaN* values (not a number) as the scaling factor and offset for channels that failed calibration.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition or impedance measurement is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>6</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>6</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>2</count>
    <item_version>0</item_version>
    <item>
      → insert the FIRST parameter entry in escaped XML here
    </item>
    <item>
      → insert the SECOND parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The currently set scaling factor and offset values are returned in the first item of the reply's parameter vector.

The payload of the server's reply holds a device-specific data entry with two parameters in its parameter list.

For each synchronously sampled analog input channel, the new scaling values calculated under the current conditions are returned in the first item of the reply's parameter vector.

Channels that failed calibration are returned in the second item of the reply's parameter vector.

If one or more channels fail calibration, the operation is still considered successful and the status field will indicate success.

The FIRST parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <offset_>
    <count>64</count>      → [The number of all channels, always 64]
    <item>24.91011</item>   → [The offset value of the first channel (channel 0)]
    <item>25.89259</item>   → [The offset value of the second channel (channel 1)]
    <item>24.945881</item>
    .
    .
    .
    <item>nan</item>        → [Channel 61 failed calibration and contains nan instead]
    <item>nan</item>        → [Channel 62 failed calibration and contains nan instead]
    <item>0</item>
  </offset_>
  <scaling_>
    <count>64</count>      → [The number of all channels, always 64]
    <item>1.001188</item>   → [The scale factor of the first channel (channel 0)]
    <item>1.001254</item>   → [The scale factor of the first channel (channel 0)]
    <item>1.001191</item>
    .
    .
    .
    <item>nan</item>        → [Channel 61 failed calibration and contains nan instead]
    <item>nan</item>        → [Channel 62 failed calibration and contains nan instead]
    <item>1</item>
  </scaling_>
</value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ structure containing two serialized C++ float arrays of size 64. For each channel, the first array (<offset_>) holds the offset values in microvolts, and the second array (<scaling_>) holds the scaling factor values. A scaling factor or an offset value at index i within the array corresponds to the i 'th channel of the device. Scaling factor and offset values are always specified for all channels that a device may provide in its full configuration, and can be ignored for unavailable channels.

If a channel fails calibration, its corresponding scaling factor and offset value will hold `nan` and the number of that channel is included in the list of channels that failed calibration (see the second parameter below).

The SECOND parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>
  <count>2</count>
  <item_version>0</item_version>
  <item>61</item>
  <item>62</item>
</value>
</boost_serialization>
```

**NOTE**

The first parameter entry is a serialized C++ `std::vector<uint16_t>`, where each item contains the zero-based number of a channel that failed calibration. In the given example, zero-based channel numbers 61 and 62 failed calibration.

Get the list of available bandpass filters (GET_BANDPASS_FILTERS)

Gets the list of available bandpass filters for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>7</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>7</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available bandpass filters is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>50</count>
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <id_>1</id_>
    <order_>3</order_>
    <sampling_rate_>250</sampling_rate_>
    <lower_cut_off_frequency_>0.100000000000000001</lower_cut_off_frequency_>
    <upper_cut_off_frequency_>0</upper_cut_off_frequency_>
  </item>
  .
  .
  .
  <item>
    <id_>1</id_>
    <order_>8</order_>
    <sampling_rate_>500</sampling_rate_>
    <lower_cut_off_frequency_>5</lower_cut_off_frequency_>
    <upper_cut_off_frequency_>200</upper_cut_off_frequency_>
  </item>
</value>
</boost_serialization>
```

[A specific filter entry (at index 0 in this case)]



NOTE

The list of available filters is a serialized C++ `std::vector<...>`, where each entry represents a single filter structure. The filter structure has following layout:

Element	Corresponding C++ data type	Description												
id_	enum	The type of the filter. The following table lists all possible values:												
		<table><tr><th>Value</th><th>Canonical name</th><th>Description</th></tr><tr><td>0</td><td>FILTER_TYPE_CHEBYSHEV</td><td>A Chebyshev filter.</td></tr><tr><td>1</td><td>FILTER_TYPE_BUTTERWORTH</td><td>A Butterworth filter.</td></tr><tr><td>2</td><td>FILTER_TYPE_BESSEL</td><td>A Bessel filter.</td></tr></table>	Value	Canonical name	Description	0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.	1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.	2	FILTER_TYPE_BESSEL	A Bessel filter.
		Value	Canonical name	Description										
		0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.										
		1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.										
2	FILTER_TYPE_BESSEL	A Bessel filter.												
order_	double	The order of the filter.												
sampling_rate_	double	The sampling rate that the filter is designed for in hertz.												
lower_cut_off_frequency_	double	The lower cutoff frequency of the filter in hertz.												
upper_cut_off_frequency_	double	The upper cutoff frequency of the filter in hertz.												

If a filter should be used for a specific channel, the zero-based index of a particular filter entry within the list of available filters is the index of the filter to specify in the configuration structure for that channel.

Get the list of available notch filters (GET_NOTCH_FILTERS)

Gets the list of available notch filters for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>8</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>8</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available notch filters is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>4</count>
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <id_>1</id_>
    <order_>4</order_>
    <sampling_rate_>250</sampling_rate_>
    <lower_cut_off_frequency_>48</lower_cut_off_frequency_>
    <upper_cut_off_frequency_>52</upper_cut_off_frequency_>
  </item>
  .
  .
  .
  <item>
    <id_>1</id_>
    <order_>4</order_>
    <sampling_rate_>500</sampling_rate_>
    <lower_cut_off_frequency_>58</lower_cut_off_frequency_>
    <upper_cut_off_frequency_>62</upper_cut_off_frequency_>
  </item>
</value>
</boost_serialization>

```

[A specific filter entry (at index 0 in this case)]

**NOTE**

The list of available filters is a serialized C++ `std::vector<...>`, where each entry represents a single filter structure. The filter structure has following layout:

Element	Corresponding C++ data type	Description												
id_	enum	The type of the filter. The following table lists all possible values:												
		<table><tr><th>Value</th><th>Canonical name</th><th>Description</th></tr><tr><td>0</td><td><code>FILTER_TYPE_CHEBYSHEV</code></td><td>A Chebyshev filter.</td></tr><tr><td>1</td><td><code>FILTER_TYPE_BUTTERWORTH</code></td><td>A Butterworth filter.</td></tr><tr><td>2</td><td><code>FILTER_TYPE_BESSEL</code></td><td>A Bessel filter.</td></tr></table>	Value	Canonical name	Description	0	<code>FILTER_TYPE_CHEBYSHEV</code>	A Chebyshev filter.	1	<code>FILTER_TYPE_BUTTERWORTH</code>	A Butterworth filter.	2	<code>FILTER_TYPE_BESSEL</code>	A Bessel filter.
		Value	Canonical name	Description										
		0	<code>FILTER_TYPE_CHEBYSHEV</code>	A Chebyshev filter.										
		1	<code>FILTER_TYPE_BUTTERWORTH</code>	A Butterworth filter.										
2	<code>FILTER_TYPE_BESSEL</code>	A Bessel filter.												
order_	double	The order of the filter.												
sampling_rate	double	The sampling rate that the filter is designed for in hertz.												
lower_cut_off_frequency_	double	The lower cutoff frequency of the filter in hertz.												

<code>upper_cut_off_</code> <code>frequency_</code>	<code>double</code>	The upper cutoff frequency of the filter in hertz.
--	---------------------	--

If a filter should be used for a specific channel, the zero-based index of a particular filter entry within the list of available filters is the index of the filter to specify in the configuration structure for that channel.

Reset scaling and offset values for each channel (RESET_SCALING)

Resets the device's scaling values to neutral scaling values for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

In neutral scaling, all offset values are set to 0 and all scaling factors are set to 1.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition or impedance measurement is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>9</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>9</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with an empty parameter vector.

Get the impedance of all channels (`GET_IMPEDANCE`)

Measures impedances of all available synchronously sampled analog input channels at the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

This command is deprecated (use [GET_IMPEDANCE_EX](#) instead).

The server waits until all channels have been measured and returns the reply afterwards.

Electrode Cz plays a special role in impedance measurement and must always be connected.

All returned impedance values are rounded to the nearest integral kilo-ohm value. All measured values are clipped within a range of 5,000 ohm to 500,000 ohm. However, if the impedance value cannot be measured exactly, the method returns a value of 4,000 for any estimated impedance value below 5,000 ohm, and 501,000 for any estimated impedance value above 500,000 ohm. The returned list only includes available channels that yielded a finite impedance value.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition or another synchronous or asynchronous impedance measurement is currently running.

The request will fail if an invalid reference signal is measured on electrode Cz (maybe due to an unconnected electrode), which does not allow calculation of impedance values on all other channels.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>10</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>10</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of measured impedances, for all available synchronously sampled analog input channels, is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <count>32</count> → [The number of measured channels that the list contains]
    <item_version>0</item_version>
    <item class_id="1" tracking_level="0" version="0">
      <first>0</first> → [The zero-based channel number]
      <second>5000</second> → [The measured impedance value in ohm for the corresponding
channel]
    </item>
    <item>
      <first>1</first>
      <second>501000</second>
    </item>
    <item>
      <first>2</first>
      <second>501000</second>
    </item>
    .
    .
    .
    <item>
      <first>29</first>
      <second>501000</second>
    </item>
    <item>
      <first>30</first>
      <second>501000</second>
    </item>
    <item>
      <first>31</first>
      <second>501000</second>
    </item>
  </value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ `std::map<uint16_t, double>`, where each `<item>` entry contains the zero-based channel number in its key (`<first>`) and the measured impedance value of that channel in ohm in its value (`<second>`). Only available channels are measured and returned.

Start asynchronous impedance measurement (START_IMPEDANCE_MEASUREMENT)

Starts continuous impedance measurement asynchronously, and raises the [IMPEDANCE_MEASUREMENT_CALLBACK](#) event for each completed measurement cycle of the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

This command is deprecated (use [START_IMPEDANCE_MEASUREMENT_EX](#) instead).

The server returns the reply immediately. The [IMPEDANCE_MEASUREMENT_CALLBACK](#) event will be raised after each completed impedance measurement cycle until the [STOP_IMPEDANCE_MEASUREMENT](#) request is sent.

Use the <customId_> element in the client's request to pass a custom value to the [IMPEDANCE_MEASUREMENT_CALLBACK](#) event that enables the event handler to identify the registrar of the event. The server doesn't use this value for processing.

If an error occurs during impedance measurement, it is forwarded to the callback as a parameter. Errors occur in the same situations as with [GET_IMPEDANCE](#). See the documentation of [IMPEDANCE_MEASUREMENT_CALLBACK](#) for details on the behavior in case of an error, and the retrieval of the measured impedance values.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>11</command_>
  <customId_>1</customId_> → [Use the customId_ value to identify the
  registrar of the event]
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
```

```

        <item_version>0</item_version>
    </parameters_>
</value>
</boost_serialization>

```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty. Use the `<customId_>` element in the client's request to pass a custom value to the [IMPEDANCE MEASUREMENT CALLBACK](#) event that enables the event handler to identify the registrar of the event.

Server reply

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
    <session_id_>1</session_id_>
    <command_>31</command_>
    <payload_>
        → insert escaped payload message of the server's reply here
    </payload_>
    <status_>2</status_>
    <error_>0</error_>
</value>
</boost_serialization>

```

The payload of the server's reply in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
    <target_>NB-2013.05.03</target_>
    <command_>11</command_>
    <customId_>1</customId_>
    <parameters_ class_id="1" tracking_level="0" version="0">
        <count>0</count>
        <item_version>0</item_version>
    </parameters_>
</value>
</boost_serialization>

```

**NOTE**

The payload of the server's reply holds a device-specific data entry with an empty parameter vector.

Server event

The [IMPEDANCE_MEASUREMENT_CALLBACK](#) event is raised for each completed measurement cycle for the issuing client of the associated data acquisition session on success.

Stop asynchronous impedance measurement (STOP_IMPEDANCE_MEASUREMENT)

Stops continuous, asynchronous impedance measurement that was previously started with [START_IMPEDANCE_MEASUREMENT](#) for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The [IMPEDANCE_MEASUREMENT_CALLBACK](#) will not be raised anymore for the issuing client.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>12</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>12</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with an empty parameter vector.

Get available channels at the device (GET_AVAILABLE_CHANNELS)

Retrieves a Boolean flag for each possible channel that a device in its full configuration may provide. The flag indicates whether the corresponding channel is actually provided and available on the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>14</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>14</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available channels is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>64</count>           → [The maximum number of all possible channels (always 64)]
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <first>0</first>          → [The zero-based channel number]
    <second>1</second>        → [A Boolean flag indicating whether the channel is available
                                (1) or not (0)]
  </item>
  <item>
    <first>1</first>
    <second>1</second>
  </item>
  <item>
    <first>2</first>
    <second>1</second>
  </item>
  .
  .
  .
  <item>
    <first>62</first>
    <second>0</second>
  </item>
  <item>
    <first>63</first>
    <second>0</second>
  </item>
</value>
</boost_serialization>
```



NOTE

The list of available channels is a serialized C++ `std::map<uint32_t, bool>`, where each entry represents a single channel that may be provided by the device. The first element (`<first>`) of the pair contains the zero-based channel number that the current entry belongs to, while the second element (`<second>`) indicates whether the channel is actually provided and available on the device (1) or not (0).

Get the list of supported sampling rates (GET_SUPPORTED_SAMPLING_RATES)

Retrieves a list of sampling rates supported by the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>15</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>15</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of supported sampling rates is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>
    <count>2</count>      → [Two valid sampling rate values are available]
    <item_version>0</item_version>
    <item>250</item>      → [A valid sampling rate of 250 Hz]
    <item>250</item>      → [A valid sampling rate of 500 Hz]
</value>
</boost_serialization>
```

**NOTE**

The list of supported sampling rates is a serialized C++ `std::vector<double>`, where each entry represents a supported sampling rate in hertz.

Get available digital I/O channels at the device (GET_DIGITAL_IOS)

Gets information about available digital I/O channels of the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>16</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>16</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available digital I/O channels is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <count>8</count> → [The number of available digital channels]
    <item_version>0</item_version>
    <item class_id="1" tracking_level="0" version="0">
      <id_>1</id_> → [The one-based ID of the digital channel]
      <direction_>0</direction_> → [The direction enumeration (0 for digital in, 1 for digital out)]
    </item>
    <item>
      <id_>2</id_>
      <direction_>0</direction_>
    </item>
    <item>
      <id_>3</id_>
      <direction_>0</direction_>
    </item>
    <item>
      <id_>4</id_>
      <direction_>0</direction_>
    </item>
    <item>
      <id_>5</id_>
      <direction_>0</direction_>
    </item>
    <item>
      <id_>6</id_>
      <direction_>0</direction_>
    </item>
    <item>
      <id_>7</id_>
      <direction_>0</direction_>
    </item>
    <item>
      <id_>8</id_>
      <direction_>0</direction_>
    </item>
  </value>
</boost_serialization>

```



NOTE

The list of available digital I/O channels is a serialized C++ `std::vector<...>`, where each entry represents an available digital I/O channel. The digital I/O channel structure has the following layout:

Element	Corresponding C++ data type	Description									
id_	uint32_t	The one-based ID of the digital I/O channel.									
direction_	enum	The direction of the channel. The following table lists valid values:									
		<table><tr><th>Value</th><th>Canonical name</th><th>Description</th></tr><tr><td>0</td><td>FILTER_TYPE_CHEBYSHEV</td><td>A Chebyshev filter.</td></tr><tr><td>1</td><td>FILTER_TYPE_BUTTERWORTH</td><td>A Butterworth filter.</td></tr></table>	Value	Canonical name	Description	0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.	1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.
Value	Canonical name	Description									
0	FILTER_TYPE_CHEBYSHEV	A Chebyshev filter.									
1	FILTER_TYPE_BUTTERWORTH	A Butterworth filter.									

Get the list of supported radio channels (GET_SUPPORTED_NW_CHANNELS)

Retrieves the available radio channels that a g.Nautilus device can use to pair its headset and base station.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>17</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>17</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of available radio channels is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>
    <count>15</count>      → [The number of available radio channels]
    <item_version>0</item_version>
    <item>11</item>        → [The number of a specific available radio channel]
    <item>12</item>
    <item>13</item>
    .
    .
    .
    <item>23</item>
    <item>24</item>
    <item>25</item>
</value>
</boost_serialization>

```

**NOTE**

The list of available radio channels is a serialized C++ `std::vector<uint32_t>`.

Get the list of supported sensitivities (GET_SUPPORTED_SENSITIVITIES)

Retrieves the supported sensitivity values for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>18</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>18</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of supported sensitivities is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>
    <count>6</count>           → [The number of supported sensitivity values]
    <item_version>0</item_version>
    <item>2250000</item>       → [A specific supported sensitivity value]
    <item>1125000</item>
    <item>750000</item>
    <item>562500</item>
    <item>375000</item>
    <item>187500</item>
</value>
</boost_serialization>
```



NOTE

The list of supported sensitivity values is a serialized C++ `std::vector<uint32_t>`.

Get the list of supported input signals (GET_SUPPORTED_INPUT_SOURCES)

Retrieves the supported input sources for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>19</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>19</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of supported input sources is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>4</count>
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <first>0</first>
    <second>EEG</second>
  </item>
  <item>
    <first>1</first>
    <second>SHORTCUT</second>
  </item>
  <item>
    <first>5</first>
    <second>TEST_SIGNAL</second>
  </item>
</value>
</boost_serialization>
```



NOTE

The list of supported input sources is a serialized C++ `std::map<int32_t, std::string>`, where each entry represents a supported input source with the identifier in its key (`<first>`) and a description in its value (`<second>`). The identifier can be used in the corresponding field of the device configuration to select one of the supported input signals.

Get information about the device (GET_DEVICE_INFO)

Retrieves extended information about the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>20</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the serve's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <target_>NB-2014.01.01</target_>
    <command_>20</command_>
    <customId_>0</customId_>
    <parameters_ class_id="1" tracking_level="0" version="0">
      <count>1</count>
      <item_version>0</item_version>
      <item>
        → insert the parameter entry in escaped XML here
      </item>
    </parameters_>
  </value>
</boost_serialization>
```

The parameter entry XML markup

```
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>
NUM_MODULES=4
SERIAL_BS=NB-2014.01.01
SERIAL_HS=NA-2014.01.01
ADDR_BS=4355
ADDR_HS=4354
FW_VER_BS=000.001.007
FW_VER_HS=000.001.007
FW_VER_A7=000.000.008
DEVICE_VER_BS=1.1.1
DEVICE_VER_HS=1.1.1
GRID_ID=02_V2.0
PCB_VER_BS=1.2
PCB_VER_HS_A2=1.2
PCB_VER_HS_A7=1.2
ENCLOSURE_VER_BS=1.1
ENCLOSURE_VER_HS=1.1
</value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ *std::string* containing extended device information.

Get the impedance of all channels (GET_IMPEDANCE_EX)

Measures impedances of all available synchronously sampled analog input channels at the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The server waits until all channels have been measured and returns the reply afterwards.

Electrode Cz plays a special role in impedance measurement and must always be connected. The request will not fail if an invalid reference signal is measured on electrode Cz (maybe due to an unconnected electrode). Instead, appropriate error codes will be assigned to the other channels.

All returned impedance values are rounded to the nearest integral ohm value in range. All measured values are clipped within a range of 600 ohm to 500,000 ohm. Negative values represent different error codes. The returned list only includes available channels that yielded a finite impedance value.

The following table lists the recommended classification of returned impedances for active gel electrodes:

Impedance value in ohm	Recommended classification for active gel electrodes
< 65,000	Good impedance.
65,000...200,000	Medium impedance.
> 200,000	Bad impedance.

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Returned value (error code)	Description
-1	The amplitude of the measured signal almost equals the original signal. The impedance is estimated to be very low.
-2	The impedance could not be calculated but is estimated to be very low.
-3	The amplitude of the measured signal is too small. The impedance is estimated to be very high.
-4	The offset of the measured signal is too large.
-7	The shape of the measured signal does not match the expected one.
-8	Too many invalid data frames were acquired during impedance measurement. This indicates bad wireless connection between the g.Nautilus headset and base station.
-9	The amplitude of the measured signal at electrode Cz is too small.
-10	The offset of the measured signal at electrode Cz is too big.
-11	The shape of the measured signal at electrode Cz does not match the expected one.
-12	Too many invalid data frames were acquired for electrode Cz during impedance measurement. This indicates bad wireless connection between the g.Nautilus headset and base station.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

The request will fail if data acquisition or another synchronous or asynchronous impedance measurement is currently running.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>21</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>21</command_>
  <customId_>0</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>1</count>
    <item_version>0</item_version>
    <item>
      → insert the parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

The payload of the server's reply holds a device-specific data entry with one parameter in its parameter list. The list of measured impedances, for all available synchronously sampled analog input channels, is returned in the first item of the reply's parameter vector.

The parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
  <value class_id="0" tracking_level="0" version="0">
    <count>32</count>           → [The number of measured channels that the list contains]
    <item_version>0</item_version>
    <item class_id="1" tracking_level="0" version="0">
      <first>0</first>           → [The zero-based channel number]
      <second>5000</second>      → [The measured impedance value in ohm for the corresponding
                                channel]
    </item>
    <item>
      <first>1</first>
      <second>-3</second>
    </item>
    <item>
      <first>2</first>
      <second>-3</second>
    </item>
    .
    .
    .
    <item>
      <first>29</first>
      <second>-3</second>
    </item>
    <item>
      <first>30</first>
      <second>-3</second>
    </item>
    <item>
      <first>31</first>
      <second>-3</second>
    </item>
  </value>
</boost_serialization>
```



NOTE

The parameter entry is a serialized C++ `std::map<uint16_t, double>`, where each `<item>` entry contains the zero-based channel number in its key (`<first>`) and the measured impedance value of that channel in ohm in its value (`<second>`). Only available channels are measured and returned.

Start asynchronous impedance measurement (START_IMPEDANCE_MEASUREMENT_EX)

Starts continuous impedance measurement asynchronously, and raises the [IMPEDANCE_MEASUREMENT_CALLBACK](#) event for each completed measurement cycle of the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

The server returns the reply immediately. The [IMPEDANCE_MEASUREMENT_CALLBACK](#) event will be raised after each completed impedance measurement cycle until the [STOP_IMPEDANCE_MEASUREMENT](#) request is sent.

Use the <customId_> element in the client's request to pass a custom value to the [IMPEDANCE_MEASUREMENT_CALLBACK](#) event that enables the event handler to identify the registrar of the event. The server doesn't use this value for processing.

If an error occurs during impedance measurement, it is forwarded to the callback as a parameter. Errors occur in the same situations as with [GET_IMPEDANCE_EX](#). See the documentation of [IMPEDANCE_MEASUREMENT_CALLBACK](#) for details on the behavior in case of an error, and the retrieval of the measured impedance values.

The request will fail if the specified session ID is not associated with any existing data acquisition session, or if the specified target device name refers to a device that is not part of the data acquisition session associated with the specified session ID.

Client request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>1</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of client's network API message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>22</command_>
  <customId_>1</customId_> → [Use the customId_ value to identify the registrar of the event]
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

This device-specific operation does not require any further parameters. Thus, the parameter vector is empty. Use the `<customId_>` element in the client's request to pass a custom value to the [IMPEDANCE MEASUREMENT CALLBACK](#) event that enables the event handler to identify the registrar of the event.

Server reply

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>1</session_id_>
  <command_>31</command_>
  <payload_>
    → insert escaped payload message of the server's reply here
  </payload_>
  <status_>2</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the server's reply in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2013.05.03</target_>
  <command_>22</command_>
  <customId_>1</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>0</count>
    <item_version>0</item_version>
  </parameters_>
</value>
</boost_serialization>
```

**NOTE**

The payload of the server's reply holds a device-specific data entry with an empty parameter vector.

Server event

The [IMPEDANCE MEASUREMENT CALLBACK](#) event is raised for each completed measurement cycle for the issuing client of the associated data acquisition session on success.

3.5.3 ASYNCHRONOUS EVENT MESSAGES

The following table lists all asynchronous event messages that the server can send for g.Nautilus devices only. The command value in the command field of the message's XML string identifies the event and is given in the table as well.

Command Value	Canonical name	Description
13	IMPEDANCE_MEASUREMENT_CALLBACK	Impedance measurement callback.

A detailed description of these messages follows.

Impedance measurement callback (IMPEDANCE_MEASUREMENT_CALLBACK)

The event that is raised when an asynchronous impedance measurement cycle has completed for the specified g.Nautilus device in the data acquisition session associated with the specified session ID.

Errors occur in the same situations as with [GET IMPEDANCE EX/GET IMPEDANCE](#). If an error occurs during impedance measurement, the server sets the corresponding parameter flag of the callback event. Another parameter flag will indicate whether the error was crucial for impedance measurement, so that it had to be stopped automatically. In this case, impedance measurement is already stopped and can be started again with another [START IMPEDANCE MEASUREMENT](#) request. If the error was not crucial for impedance measurement, only the current measurement cycle that the callback was raised for is affected, and impedance measurement continues.

The server sets the <customId_> element in the event's device-specific data entry to the <customId_> value that the client specified in its corresponding [START IMPEDANCE MEASUREMENT](#) request.

See the [GET IMPEDANCE EX/GET IMPEDANCE](#) request for details on impedance measurement and interpretation of the measured values.

Event

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <session_id_>0</session_id_>
  <command_>32</command_>
  <payload_>
    → insert escaped payload message of the client request here
  </payload_>
  <status_>0</status_>
  <error_>0</error_>
</value>
</boost_serialization>
```

The payload of the message in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <target_>NB-2014.01.01</target_>
  <command_>13</command_>
  <customId_>1</customId_>
  <parameters_ class_id="1" tracking_level="0" version="0">
    <count>4</count>
    <item_version>0</item_version>
    <item>
      → insert the FIRST parameter entry in escaped XML here
    </item>
    <item>
      → insert the SECOND parameter entry in escaped XML here
    </item>
    <item>
      → insert the THIRD parameter entry in escaped XML here
    </item>
    <item>
      → insert the FOURTH parameter entry in escaped XML here
    </item>
  </parameters_>
</value>
</boost_serialization>
```



NOTE

The payload of the event holds a device-specific data entry with four parameters in its parameter list.

The measured impedances for all available synchronously sampled analog input channels are returned in the first item of the event's parameter vector.

A flag indicating whether an error has occurred during the last impedance measurement cycle is returned in the second item of the event's parameter vector.

A flag indicating whether an occurred error has been crucial for impedance measurement so that asynchronous measurement had to be stopped automatically, if any, is returned in the third item of the event's parameter vector.

The error message describing an occurred error, if any, is returned in the fourth item of the event's parameter vector.

The server sets the `<customId_>` element in the event's device-specific data entry to the `<customId_>` value that the client specified in its corresponding

[START IMPEDANCE MEASUREMENT](#) request.

The FIRST parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value class_id="0" tracking_level="0" version="0">
  <count>32</count>
  <item_version>0</item_version>
  <item class_id="1" tracking_level="0" version="0">
    <first>0</first>
    <second>5000</second>
  </item>
  <item>
    <first>1</first>
    <second>-3</second>
  </item>
  <item>
    <first>2</first>
    <second>-3</second>
  </item>
  .
  .
  .
  <item>
    <first>29</first>
    <second>-3</second>
  </item>
  <item>
    <first>30</first>
    <second>-3</second>
  </item>
  <item>
    <first>31</first>
    <second>-3</second>
  </item>
</value>
</boost_serialization>
```



NOTE

The first parameter entry is a serialized C++ `std::map<uint16_t, double>`, where each `<item>` entry contains the zero-based channel number in its key (`<first>`) and the measured impedance value of that channel in ohm in its value (`<second>`). Only available channels are measured and returned.

The payload of the network API message holds a device specific data entry. The parameter vector of the device specific data entry holds a list of channel / impedance pairs (i.e. a serialized C++ `std::map<uint16_t, double>`).

The SECOND parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>0</value>
</boost_serialization>
```

**NOTE**

The second parameter entry is a serialized C++ bool value indicating whether an error has occurred (1) during the last impedance measurement cycle that this callback event has been raised for, or not (0).

The THIRD parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value>0</value>
</boost_serialization>
```

**NOTE**

The third parameter entry is a serialized C++ bool value indicating whether an occurred error was crucial for impedance measurement, so that an asynchronous impedance measurement had been stopped automatically by the server.

If the value equals 1, impedance measurement had been stopped automatically and can be started again with another [START IMPEDANCE MEASUREMENT](#) request.

Otherwise, the value equals 0 and the error was not crucial for impedance measurement. Then, only the current measurement cycle that the callback was raised for is affected, and impedance measurement continues.

If no error has occurred, this value can be ignored.

The FOURTH parameter entry in XML markup

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE boost_serialization>
<boost_serialization signature="serialization::archive" version="19">
<value></value>
</boost_serialization>
```

**NOTE**

The fourth parameter entry is a serialized C++ `std::string` containing the message describing an occurred error. If no error has occurred, this value will be an empty string (like in the example above).

g.NEEDACCESS
NETWORK ENABLED EASY DATA ACCESS